Distributed Parallel Hyperspectral Unmixing for Large-Scale Data in Spark Environments via Geometric Distance

Carlos Cañada, Mercedes E. Paoletti, *Senior Member, IEEE*, María B. García-Flores, Xuanwen Tao, *Member, IEEE*, Rafael Pastor-Vargas, *Senior Member, IEEE*, and Juan M. Haut, *Senior Member, IEEE*

***Abstract*—Hyperspectral unmixing addresses the challenge of mixed pixels in hyperspectral images by identifying the number of pure pixels (endmembers), extracting their spectral signatures, and estimating their proportions (abundances) in each pixel composing the scene. Traditional hyperspectral unmixing methods often struggle with scalability and computational efficiency when dealing with gigabyte-scale datasets. In this article, we propose a distributed parallel geometric distance (DPGD) method for hyperspectral unmixing, exploring the computational power and benefits of distributed parallel processing within a distributed computing framework. The proposed DPGD leverages geometric distance measurements to accurately identify endmembers and estimate their abundances, taking into account the intrinsic similarities within hyperspectral images. This provides a clearer representation of the data structure, leading to improved unmixing accuracy. By using the Spark programming model, the computational workload is efficiently distributed across multiple nodes, significantly reducing processing time. Experimental results on real hyperspectral datasets demonstrate that DPGD scales effectively up to 32 nodes and 290.9 GB of data, achieving competitive accuracy and efficiency compared to state-of-the-art methods. The code is available at https://github.com/ccaadaro/DPDG**

***Index Terms*—Big data, distributed parallel geometric distance (DPGD), hyperspectral unmixing.**

I. Introduction

DUE to advancements in airborne and satellite sensing technology conducted in the past five decades and the increasing number of Earth observation (EO) missions, the field of remote sensing has experienced an unprecedented increase in the volume of data captured in recent years [1]. For instance, the Sentinel satellites deployed by European Space Agency (ESA) are generating over 25 TB of new data per day, providing an unprecedented overview of the Earth and its phenomena [2]. Coupled with the increasing amounts of EO missions, this growth in remote sensing data is also driven by improvements in spatial, spectral, radiometric, and temporal resolutions, as well as by increased sensor data acquisition rates, producing massively highly detailed and nuanced images. The implications of these advances are profound. Better spatial resolution improves the detection of smaller features on the surface, which facilitates detailed object analysis, although it introduces a larger number of pixels per scene, each with its own spectral information. Also, higher spectral resolution captures detailed information about surface composition across numerous spectral bands, from visible to short-wave infrared (SWIR) wavelengths, enhancing the distinction of subtle material differences, but it introduces exponential complexity due to increasingly higher dimensional data. Higher temporal resolution yields more frequent observations of the same area, supporting a better understanding of dynamical/repetitive processes while increasing the influx of data to be handled and processed. Furthermore, higher radiometric resolution enhances sensitivity to subtle variations in reflected or emitted energy, yielding pixels with more precise intensity gradations and nuanced characteristics. However, this increase in bit depth significantly enlarges the data size and storage demands. Thus, the increase in the quality and number of data is dual-faceted. On the one hand, they facilitate the observation and analysis of the Earth's surface with an unprecedented level of detail and timeliness. Consequently, more informed and timely responses could be produced to environmental monitoring [3], disaster management [4], [5], and providing an excellent tool to perform spatial analysis [6]. Nonetheless, the increasing volume of data requires the development of optimized and efficient methods to extract relevant knowledge in a proper amount of time, intensifying demands on computing infrastructures and high-performance processing strategies [7], [8].

Building on this trend, hyperspectral image (HSI) have emerged as valuable sources of knowledge for remote sensing

Received 7 April 2025; revised 1 August 2025 and 2 October 2025; accepted 15 October 2025. Date of publication 23 October 2025; date of current version 7 November 2025. This work was supported in part by the I+D+i Project through MICIU/AEI/10.13039/501100011033 and Fondo Europeo de Desarrollo Regional (FEDER)/Unión Europea (UE) under Grant PID2023-149880NA-I00, in part by European Regional Development Fund (ERDF) of European Union Interreg V-A Espana-Portugal (POCTEP) 2021–2027 Program under Grant 0206_RAT_EOS_PC_6_E, and in part by the 2024 Leonardo Grant for Scientific Research and Cultural Creation from the Banco Bilbao Vizcaya Argentaria (BBVA) Foundation. *(Corresponding authors: Juan M. Haut; Rafael Pastor-Vargas.)*

Carlos Cañada, Mercedes E. Paoletti, María B. García-Flores, and Juan M. Haut are with the Department of Technology of Computers and Communications, University of Extremadura, 10003 Cáceres, Spain (e-mail: juanmariohaut@unex.es).

Xuanwen Tao is with the Imec-Vision Lab, Department of Physics, University of Antwerp, 2000 Antwerp, Belgium.

Rafael Pastor-Vargas is with the Department of Communication Systems and Control, National University of Distance Education, 28040 Madrid, Spain (e-mail: rpastor@scc.uned.es).

Digital Object Identifier 10.1109/TGRS.2025.3624287

1558-0644 © 2025 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 09,2026 at 11:45:48 UTC from IEEE Xplore. Restrictions apply.

applications. In particular, HSIs have attracted considerable attention within the remote sensing community due to their rich and detailed spectral information and their versatility to be applied to a wide array of remote sensing challenges. These images are collected by imaging spectrometers, i.e., sensors designed to detect photons across a broad portion of the electromagnetic spectrum, typically ranging in 400–2500 nm, from the visible (VIS) through the near infrared (NIR) or SWIR. These sensors capture the photon energy and translate it into electric signals, which are recorded as hundreds of narrow, continuous spectral bands and arranged into spatial dimensions. The resulting product is a hyperspectral data cube, in which two dimensions represent spatial information, i.e., the arrangement of the pixels, and the third dimension contains spectral information, i.e., the content of the pixels. Thus, by using the unique interactions between materials and light, the spectral information of each pixel is a key asset for accurate identification of the components included in a remotely sensed scene through the analysis of their geo-biophysical and geo-biochemical variables. The importance of HSIs has led to significant efforts to enhance spectrometer sensitivity, with advancements in optical components, signal-to-noise ratio (SNR) optimization, and detector technology, among other areas [9].

Focusing on satellite-borne instruments, the advances in sensor technology have increased the availability of high-quality HSI data, providing scenes with a spatial resolution of $\ll 30$ mpp. Some remarkable examples are the successful deployment of the Hyperspectral Precursor of the Application Mission (PRISMA) by the Agenzia Spaziale Italiana (ASI); the DLR Earth Sensing Imaging Spectrometer and the environmental mapping and analysis program (EnMAP) by the German Aerospace Center (DLR); the hyperspectral imager suite (HISUI) operated by Japanese Ministry of Economy, Trade, and Industry (METI); or the Earth Surface Mineral Dust Source Investigation (EMIT) developed at NASA's Jet Propulsion Laboratory (JPL). These instruments are milestones in the spectral observation of the Earth's surface, paving the way for new and more sophisticated sensors, such as the PRISMA second generation (PSG) [10] or the Copernicus hyperspectral imaging mission for the environment (CHIME) driven by the ESA [11]. Furthermore, technical advances such as the use of narrowband thin-film interference filters [12] are promoting the development of smaller, compact, and cost-effective sensing instruments, which are deployed within small satellites (nanosats, CubeSats, and so on). These improvements open new opportunities for low-cost, high-frequency, and scalable HSI missions, such as the enhanced light offner imaging spectrometer (ELOIS) initiative [13] or the cubesat constellation proposed by the compact smartspectral imager for monitoring bioagricultural areas (CSIMBAs) [14]. Moreover, miniaturized platforms enable the development of flexible mission configurations with small revisit times, facilitating the acquisition of large volumes of HSI data.

In this context, not only the high data acquisition rates but also the large spectral dimensionality of HSI data compound the challenge of developing fast, efficient, and scalable processing methods. With respect to spectral features, the acquisition of hundreds of contiguous spectral bands introduces high spectral variability and redundancy due to illumination conditions, atmospheric effects, and sensor noise, among other factors. As a result, the complexity of data modeling and interpretation is greatly compounded to encompass a wide range of spectral variations to ensure accurate analysis.

This spectral complexity translates into a considerable computational burden, particularly for pixel-wise tasks such as classification, target detection, and spectral unmixing. In the case of unmixing applications, the problem is particularly acute. Due to intrinsic limitations in data acquisition, each spectral pixel often spans several square meters of the observed surface, integrating the spectral contributions of multiple materials present within the pixel. Consequently, the observed spectra are typically mixed, making it challenging to extract meaningful and physically interpretable information from the HSI scene. The literature offers a wide range of approaches to address the inherently ill-posed problem of spectral mixing [15]. These methods generally aim to decompose each mixed pixel into a set of constituent pure spectral components (endmembers) and their corresponding abundance fractions. Among these methodologies, geometric distance-based approaches have gained increasing attention for their interpretability and effectiveness [16], [17]. These methods exploit the geometric structure of the spectral space, leveraging the assumption that the mixed pixels lie within a simplex formed by the endmembers. By quantifying and maximizing geometric criteria, these techniques attempt to estimate the vertices (endmembers) that best explain the observed spectral variability. Notable contributions in this direction maximum distance analysis (MDA) [18] and its geometry-based counterpart (NMDA) [19], which proposed unmixing strategies based on distance analysis and geometric distance metrics, respectively. These methods identify optimal endmember sets by selecting spectral signatures that are geometrically well-separated, thus improving the robustness and accuracy of the unmixing process in the presence of noise or spectral variability.

Nevertheless, these methods usually involve iterative optimization algorithms that operate under sparsity, statistical, or geometric constraints and often handle large-scale spectral matrices. The high dimensionality of HSI data further exacerbates the problem, introducing the well-known "curse of dimensionality," which demands larger training datasets, more complex models, and advanced regularization techniques to achieve reliable performance. These factors contribute to high algorithmic complexity and substantial computational overhead. Processing time typically increases nonlinearly with the number of spectral bands and spatial resolution, while memory usage grows rapidly, as each HSI scene often ranges from hundreds of megabytes to several gigabytes when it is stored in floating-point format. Loading such volumes of data into memory can quickly exhaust the resources of standard computing systems. Moreover, as the volume, spatial coverage, and diversity of HSI data continue to expand, these computational and storage requirements become critical bottlenecks.

To address these computational and memory constraints, big data technologies have emerged as a powerful and scalable

Authorized licensed use limited to: Hohai University Library. Downloaded on November 09,2026 at 11:45:48 UTC from IEEE Xplore. Restrictions apply.

solution for HSI data analysis, enabling efficient parallel processing and distributed storage over large-scale computing infrastructures [20], [21]. In particular, frameworks such as Apache Spark provide in-memory computing capabilities, automatic fault tolerance, and flexible data abstraction mechanisms like resilient distributed datasets (RDDs) and DataFrames, which are particularly well-suited for the high-throughput processing demands of HSI data. By partitioning large HSI cubes into smaller blocks and distributing these partitions across multiple worker nodes, Spark facilitates the concurrent execution of complex unmixing algorithms. This parallelization dramatically reduces processing times and improves scalability, making it feasible to apply computationally intensive techniques to very large datasets. Moreover, these distributed approaches improve robustness and enable real-time or near-real-time performance for fast decision-making applications. As the volume and velocity of HSI data continue to grow, the integration of unmixing methods with big data frameworks represents a critical step toward making large-scale, high-fidelity hyperspectral analysis practically viable.

A. Contributions

With the aim of conducting efficient and effective hyperspectral unmixing, this article presents a new distributed parallel geometric distance (DPGD) method for hyperspectral unmixing. The key contributions of the proposed DPGD can be summarized as follows.

1) The proposed DPGD method employs geometric distance measurements to directly and accurately identify endmembers and estimate their abundances. This innovative method improves upon traditional hyperspectral unmixing techniques by leveraging the intrinsic similarities within hyperspectral data, leading to enhanced accuracy in endmember extraction.
2) Furthermore, inspired by the cloud computing paradigm, the proposed DPGD employs the flexible Spark programming model to distribute the computational workload across multiple nodes. This significantly reduces processing time and increases computational efficiency, enabling the effective analysis of replicated hyperspectral datasets up to 290.9 GB in size. The DPGD method demonstrates the advantages of parallelization for processing hyperspectral data, outperforming existing methods in scalability and speed.

This article is organized as follows. First, Section II presents a comprehensive review of current HSI missions, together with the adoption of high-performance computing (HPC) techniques to improve the efficiency of processing methods. It also outlines the background of several related works, with a particular focus on the key characteristics of the MDA and NMDA algorithms, highlighting their respective strengths and the limitations that hinder their performance. Section III provides the description of the main contribution of this work, i.e., the basic concepts of parallelism and their application in the design of DPGD. Section IV validates the effectiveness and efficiency of the proposed algorithm through extensive experimentation. Finally, Section V presents the conclusions, summarizing the new findings and proposing future research lines.

II. Delving Into Hyperspectral Unmixing: Challenges and Related Works

This section provides a comprehensive overview of the unmixing problem from a big data-based point of view, including the characteristics of current hyperspectral missions, the computational and storage constraints of unmixing at scale, and a structured review of classical algorithmic families, including geometric, statistical, and sparse regression methods. We also highlight the importance of leveraging modern computing paradigms, such as parallel, distributed, and cloud-based frameworks, to enable efficient large-scale unmixing.

A. Hyperspectral Imagery in the Big Data Era

The current generation of EO missions, considering both airborne and satellite-based programs, has dramatically increased the availability and resolution of hyperspectral imagery. Airborne sensors such as the classic airborne visible/infrared imaging spectrometer (AVIRIS-C) and its next-generation counterparts, AVIRIS-NG and AVIRIS-3, have progressively improved the spectral and spatial resolutions [22], [23], [24]. AVIRIS-C operates in the range 380–2500 nm, capturing the spectral radiance in 224 contiguous channels with 10 nm of spectral resolution, 20 m of spatial resolution, and 12 bits of radiometric resolution. Each scan delivers 677 swath pixels, producing up to 10 GB per flight. AVIRIS-NG uses a wider spectral range with improved SNR, operating in the 380–2510-nm wavelengths, capturing the spectral radiance in 480 contiguous channels with 5 nm of spectral resolution, 0.3–20 m of spatial resolution, and 14 bits of radiometric resolution. Each scan delivers 600 swath pixels, producing up to 1.0 TB of raw data. Finally, AVIRIS-3 works in the range 380–2510 nm, recording 1240 swath pixels with 284 spectral bands, 7.5 nm of spectral resolution, and 0.5–20 m of spatial resolution.

In a similar way, orbital missions offer continuous global coverage with hundreds of bands and resolutions down to 30 m. For instance, PRISMA [25] follows a sun-synchronous orbit with a repeat cycle of 29 days. It is equipped with a panchromatic sensor (PAN) and a medium-resolution hyperspectral camera (HYC), with a data acquisition rate of up to 600 Mb/s. The HYC sensor works in both the VIS/NIR interval (400–1010 nm) and the NIR/SWIR range (920–2505 nm), producing 66 and 173 bands, respectively, with a spectral resolution of $\leq$ 12 nm, 30 m of spatial resolution, 12 bits of radiometric quantization, and a pixel resolution of 1000 pixels. Also, EnMAP [26] follows a sun-synchronous orbit with a repeat cycle of 27 days. It is equipped with a high-resolution dual HSI, covering the VNIR (418.2–993.0 nm) and SWIR (902.2–2445.5 nm) regions. These sensors produce between 91 and up to 163 channels with spectral resolutions of 6.4 and 10.0 nm on average, respectively, 30 m of spatial resolution, 14 bits of radiometric resolution, and 1000 valid pixels. Moreover, future EO missions are planned to acquire high volumes of HSIs, such as CHIME [27], which

Authorized licensed use limited to: Hohai University Library. Downloaded on November 09,2026 at 11:45:48 UTC from IEEE Xplore. Restrictions apply.

will expand the current suite of Sentinel missions. Its two sun-synchronous satellites will be equipped with a HSI to acquire 200 bands in the range 400–2500 nm with ≤ 10 nm of spectral resolution and 30 m of spatial resolution.

On the other hand, the miniaturization of space technologies has enabled the deployment of small satellites, including CubeSats and nanosatellites, equipped with increasingly capable hyperspectral payloads that are able to prove large amounts of HSI data at considerably lower costs and shorter timescales [28]. Some promising projects are the aforementioned ELOIS [13] and CSIMBA [14], which can acquire more than 5 TB of data each day or the polar-orbiting cubesat proposed by the hyperspectral bridge for Sentinels (HYBRISs) project [29], which will provide concurrent data in space and time to that provided by Sentinel 2 (multispectral data) and Sentinel 3, in particular that data collected by the Ocean and Land Color Instrument (OLCI). Also, the hyperspectral small satellite for ocean observation (HYPSO) [30] is a CubeSat mission of Norwegian University of Science and Technology to observe ocean color along the coast of Norway. These platforms offer a flexible and low-cost alternative for frequent and targeted observations, and are especially useful in applications such as precision agriculture, water quality monitoring, urban mapping, and environmental monitoring.

In this context, it is noteworthy that the rapid increase in EO missions, coupled with significant advancements in sensor technology, has resulted in an unprecedented expansion in hyperspectral and remote sensing data volumes. These advances collectively define a new data-intensive paradigm known as remote sensing big data (RSBD) [31], [32], which demands advanced data processing, storage, and analysis frameworks to effectively manage vast and complex datasets. The defining characteristics of RSBD include traditional aspects of big data, i.e., huge data volume, large data variety, and high data acquisition/generation velocity, as well as some specific features of remote sensing data, such as multisource, multiscale, high-dimensional, dynamic-state, isomer, and nonlinearity characteristics [33]. In this sense, HSIs almost meet them all, where different instruments, with different scales of observation, including spectral, spatial, radiometric, and time-varying properties, are currently generating high-dimensional data cubes over a wide and varied area of observation at different times.

B. Cloud-Driven Parallelism in Hyperspectral Unmixing

The increasing volume, dimensionality, and complexity of HSIs acquired by modern remote sensing missions have driven a growing adoption of HPC techniques in HSI processing.

In particular, tasks such as classification, anomaly detection, and unmixing are computationally demanding due to the high dimensionality of HSIs, which leads to data sparsity and increases the amount of information required to generalize effectively, which is compounded by the adoption of iterative optimization processes that can be computationally demanding [34]. Among these tasks, hyperspectral unmixing is one of the most computationally intensive [35].

The research community has proposed different hyperspectral unmixing strategies aimed not only at improving accuracy but also at enhancing robustness, computational efficiency, and scalability to meet the demands of real-world remote sensing applications. These methods are typically developed under two main physical models, i.e., the linear mixing model (LMM) and the nonlinear mixing model (NLMM).

Both models can be expressed in the general form of (1). Here, $\mathbf{x} \in \mathbb{R}^M$ denotes the spectral vector observed at a pixel, with M spectral bands. This sample can be modeled as a combination of K pure spectral signatures (endmembers) $\{\mathbf{s}_k \in \mathbb{R}^M\}_{k=1}^K$ and their corresponding abundance coefficients $\{a_k \in \mathbb{R}\}_{k=1}^K$, through a mixing function $f(\cdot)$

$$\mathbf{x} = f(\mathbf{S}, \mathbf{a}) \tag{1}$$

where $\mathbf{S} = [\mathbf{s}_1, \ldots, \mathbf{s}_K] \in \mathbb{R}^{M \times K}$ is the matrix of endmembers and $\mathbf{a} = [a_1, \ldots, a_K]^\top \in \mathbb{R}^K$ is the abundance vector.

However, while the NLMM assumes that interactions between materials lead to nonlinear combinations of endmember spectra, the LMM considers the observed pixel spectrum as a linear combination of a set of endmembers, each weighted by its abundance fraction [36], [37]. Consequently, (1) is redefined as a simple weighted linear combination of endmembers

$$\mathbf{x} = a_1\mathbf{s}_1 + \cdots + a_k\mathbf{s}_k = \sum_{k=1}^{K} a_k\mathbf{s}_k = \mathbf{Sa}. \tag{2}$$

Due to its simplicity, a wide range of unmixing algorithms have been developed within the LMM framework. These can be broadly categorized into three families: statistical-, sparse regression-, and geometric-based approaches. Statistical unmixing methods approach the problem from a probabilistic or generative perspective, primarily relying on parameter estimation techniques to extract endmembers and their corresponding abundance fractions. As such, they offer a flexible framework capable of capturing endmember variability and nonlinear mixing effects. However, this modeling flexibility often comes at the cost of increased computational complexity. Popular statistical techniques include independent component analysis (ICA) [38], dependent component analysis (DECA) [39], the maximization of non-Gaussianity (MaxNG) [40], and Bayesian approaches [41], among others.

On the other hand, sparse regression-based approaches assume that each observed pixel spectrum can be represented as a sparse linear combination of a small subset of pure spectral signatures known in advance from a spectral library. This enables semi-supervised unmixing, where the goal is to identify the optimal set of elements from the library that best reconstruct each pixel. Widely used techniques in this family include sparse unmixing by variable splitting and augmented lagrangian (SUnSAL) [42], SUnSAL with total variation regularization (SUnSAL-TV) [43], and collaborative sparse unmixing methods such as CLSUnSAL [44] or the CSUnL0 [45]. Other methods incorporate spatial or structural priors, such as structured sparse regularized nonnegative matrix factorization (SS-NMF) [46] and superpixel-based low-rank regularization (SCLRSU) [47]. While these methods provide increased flexibility and adaptability to diverse scenes, they often face scalability challenges when dealing with large spectral libraries or high-resolution HSI cubes.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 09,2026 at 11:45:48 UTC from IEEE Xplore. Restrictions apply.

Finally, geometric unmixing methods work under the assumption that each observed pixel lies within a convex combination (simplex) of a set of endmembers. These approaches are typically divided into two main categories: pure pixel-based and minimum volume-based methods [48], [49]. Pure pixel-based algorithms assume that each endmember is represented by at least one pure pixel in the dataset, corresponding to a spectral vector located at a vertex of the data simplex. Due to their low computational requirements and intuitive geometric interpretation, they are among the most popular techniques in practical applications. Prominent examples include N-FINDR [50], pixel purity index (PPI) [51], and vertex component analysis (VCA) [52], which seeks to iteratively identify extreme points in the data cloud. In contrast, minimum volume-based methods relax the pure pixel assumption by estimating a mixing matrix, the associated simplex of which encloses the data while minimizing its volume. As a result, these methods are better suited for highly mixed scenes, although at the cost of increased computational complexity. Representative techniques include minimum volume enclosing simplex (MVES) [53], simplex identification via variable splitting and augmented lagrangian (SISAL) [36], minimum volume simplex analysis [54], and minimum volume constrained nonnegative matrix factorization (MVC-NMF) [55], [56], which imposes volume minimization to regularize the solution space. In this context, the MDA [18], [19] is an interesting geometric method that identifies endmembers by iteratively selecting the most spectrally dissimilar pixels based on pairwise distances.

Focusing on geometric approaches, these methods are often less sensitive to initialization and can be computationally more efficient in low-dimensional spaces compared to purely statistical or learning-based methods. Nevertheless, their performance tends to degrade in high-dimensional settings because of the curse of dimensionality and, more critically, the rapidly growing volume and resolution of hyperspectral data. As modern sensors generate increasingly large and complex datasets, geometric methods face significant computational challenges related to memory usage, processing time, and scalability, particularly when applied to real-time or large-scale remote sensing scenarios.

Notwithstanding significant advancements in hyperspectral unmixing techniques, the majority of existing geometric algorithms are not designed to handle large-scale data efficiently. Although there have been efforts to accelerate these algorithms through GPU implementations [57], [20], [58], [59], research into broader HPC strategies remains limited [21], [60].

In this context, to address the challenges posed by modern HSI, three major HPC paradigms can be considered: *parallel*, *distributed*, and *cloud computing*. Regarding parallel computing, it accelerates data processing by dividing large computational tasks among multiple cores of specialized hardware, such as field-programmable gate arrays (FPGAs), graphic processing units (GPUs), or tensor processing units (TPUs), enabling concurrent execution. These architectures are particularly well-suited to some geometric methods that involve linear algebra operations and vector projections, which can be parallelized efficiently [61], [62], [63], [64]. However, their scalability is inherently limited by the available hardware on a single node, and performance may degrade when data sizes exceed the device memory.

Distributed computing addresses single-node memory and compute limitations by spreading both data and processing tasks across multiple interconnected nodes. This model enables scalability for very large HSI cubes or high-resolution scenes.

In the context of geometric unmixing, distributed systems have been used to implement methods such as N-FINDR [65] or simplex volume optimization in parallel across spatial or spectral partitions [66], [67]. However, they face nontrivial challenges: internode communication overhead, data partitioning strategies that preserve geometric structure, and load balancing are critical concerns. Moreover, many geometric algorithms rely on global information (e.g., spectral extrema or full convex hulls), which complicates distributed execution.

Finally, cloud computing extends the benefits of parallel and distributed models by offering on-demand, scalable computing and storage resources, enabling flexible adaptation to evolving data processing needs. In addition, cloud platforms support diverse data types and processing frameworks, providing tools and services to facilitate rapid deployment, elastic resource allocation, and integration with analytics pipelines. Also, cloud service providers implement advanced security technologies and compliance measures to help users address data security and privacy concerns. Although some works explore the use of cloud computing [60], [7], there are few examples in the current literature on HSI that adapt geometric methods to such platforms. While some efforts have been made to implement classification algorithms in cloud environments [68], research on spectral unmixing methods, especially those based on geometric principles, is still limited.

C. Background Algorithms

Given the growing complexity and size of hyperspectral data, traditional geometric unmixing methods such as MDA face significant computational challenges. In particular, despite its interpretability and effectiveness, MDA iteratively computes distances to affine structures, which becomes quite expensive in high-dimensional settings. To address these limitations, the integration of MDA into a cloud-based distributed framework is explored, leveraging Apache Spark to parallelize computations across multiple nodes.

1) MDA Algorithm: Most existing algorithms require a priori the number of endmembers to extract spectral signatures. To overcome this, Tao et al. [18] proposed the maximum distance algorithm (MDA), which simultaneously determines the number of endmembers and extracts them. The MDA is based on the observation that endmembers form a simplex with the greatest volume over all pixel combinations.

Let $\mathbf{x}_i \in \mathbb{R}^M$ denote the spectrum of the ith pixel, with M spectral bands. The directed vector between two pixels $\mathbf{x}_i$ and $\mathbf{x}_j$ is defined as $\overrightarrow{\mathbf{x}_j\mathbf{x}_i} = \mathbf{x}_i - \mathbf{x}_j$. Specifically, MDA computes

Authorized licensed use limited to: Hohai University Library. Downloaded on November 09,2026 at 11:45:48 UTC from IEEE Xplore. Restrictions apply.

the distance d_{ij} between two pixel points $\mathbf{x}_i$ and $\mathbf{x}_j$ by

$$\begin{aligned} d_{ij} &= \| \mathbf{x}_i - \mathbf{x}_j \|_2 \\ &= \sqrt{\left(x_{1i} - x_{1j}\right)^2 + \cdots + \left(x_{Mi} - x_{Mj}\right)^2}. \end{aligned} \tag{3}$$

The farthest pixel point from the origin in the pixel space is selected as the first endmember, denoted as $\mathbf{x}_A$. Consequently, the pixel point farthest from $\mathbf{x}_A$ is selected as the second endmember, $\mathbf{x}_B$. Next, the MDA computes the maximum distance from any pixel point $\mathbf{x}_i$ to the line $\mathrm{L}_{\{AB\}}$ formed by the first and second endmembers using

$$d_{i_\mathrm{L}_{AB}} = \left\| \overrightarrow{\mathbf{x}_B \mathbf{x}_i} \right\|_2 \sin\alpha \tag{4}$$

where α is the angle between the two vectors $\overrightarrow{\mathbf{x}_B \mathbf{x}_i}$ and $\overrightarrow{\mathbf{x}_B \mathbf{x}_A}$. The pixel point with the maximum distance is selected as the third endmember $\mathbf{x}_C$. The three endmembers form a plane $P_{\{ABC\}}$, with $\mathbf{n}$ representing its normal vector. Then, the MDA selects the fourth endmember by computing the maximum distance between any pixel point and the plane $P_{\{ABC\}}$ by

$$d_{i_P_{\{ABC\}}} = \left\| \overrightarrow{\mathbf{x}_B \mathbf{x}_i} \right\|_2 |\cos\beta| \tag{5}$$

where β is the angle between $\overrightarrow{\mathbf{x}_B \mathbf{x}_i}$ and $\mathbf{n}$. All extracted endmembers will form an affine hull $\mathrm{Aff}\{ABCD\cdots\}$, i.e., the minimal affine subspace containing all current endmembers. Consequently, the MDA captures the farthest pixel point to the affine hull as the next endmember. The distance between any pixel point and the affine hull can be obtained by

$$d_{i_\mathrm{Aff}\{ABCD\ldots\}} = \frac{\left|\mathbf{w}^T \mathbf{x}_i + b\right|}{\|\mathbf{w}\|_2} \tag{6}$$

where $\mathbf{w}$ is the normal vector of the affine hull $\mathrm{Aff}\{ABCD\cdots\}$ and b is the offset or distance between the coordinate origin and the affine hull. The MDA concludes the endmember extraction operation when the distance between any pixel point and the affine hull is zero.

2) NMDA Algorithm: After reviewing the original MDA algorithm, it is found that even if MDA is capable of extracting the spectral signatures of endmembers and determining their number simultaneously, it still faces two major obstacles as follows.

1) MDA has to calculate many distances, such as the distance pixel-to-pixel, pixel-to-line, pixel-to-plane, and pixel-to-affine hull. This process is extremely complex and computationally demanding.
2) The normal vector and offset are required to compute the distance between any pixel point and the affine hull, which is highly expensive in high-dimensional HSIs.

To address these two problems, Tao et al. [19] proposed NMDA to improve the original MDA in two aspects. On the one hand, NMDA computes the distance between any pixel point and one specific affine hull to finish endmember extraction, providing a simpler spectral unmixing operation than MDA. On the other hand, NMDA replaces (6) in MDA by the new distance given in (7) to obtain the distance between any pixel point to the affine hull $\mathrm{Aff}\{ABCD\cdots\}$, where α is the angle between $\overrightarrow{\mathbf{x}_A \mathbf{x}_i}$ and the normal vector $\mathbf{w}$ of the affine hull $\mathrm{Aff}\{ABCD\cdots\}$

$$\begin{aligned} d_{i_P_{\{ABC\cdots\}}} &= \| \overrightarrow{\mathbf{x}_A \mathbf{x}_i} \|_2 \, |\cos\alpha| \\ &= \| \overrightarrow{\mathbf{x}_A \mathbf{x}_i} \|_2 \left| \frac{\overrightarrow{\mathbf{x}_A \mathbf{x}_i} \cdot \mathbf{w}}{\| \overrightarrow{\mathbf{x}_A \mathbf{x}_i} \|_2 \, \|\mathbf{w}\|_2} \right| \\ &= \frac{\left| \overrightarrow{\mathbf{x}_A \mathbf{x}_i} \cdot \mathbf{w} \right|}{\|\mathbf{w}\|_2}. \end{aligned} \tag{7}$$

Obviously, compared to MDA, NMDA only needs the normal vector to complete endmember extraction.

III. Methodology

To overcome the computational burden of processing high-dimensional hyperspectral data, we propose a novel distributed approach. The increasing workload of remote sensing data processing methods in general, and of HSIs in particular, makes it necessary to design new algorithms with a more committed approach to the correct exploitation and optimization of available computational resources. In particular, both the quantity and the high dimensionality of HSIs make them suitable for big data-based analysis, where a great effort has been devoted to the development of HPC strategies for remote sensing, including GPU-accelerated unmixing chains, parallel implementations of sparse coding methods, and hybrid CPU–GPU frameworks that enable large-scale or even real-time processing [20], [35], [31], [32]. Specifically, cloud computing offers a cost-effective, flexible, and scalable solution (both horizontally and vertically) for processing massive hyperspectral datasets. Despite these advances, most of the research has focused on algorithmic development in single-node or small-scale environments. Indeed, compared to current trends, the remote sensing community has devoted little effort to adapt unmixing methods or designing new algorithms that explicitly target distributed or cloud-based environments [36], [37], [21]. In contrast, the dominant research direction has shifted toward the improvement of endmember initialization, the use of large spectral libraries and sparse models [69], multimodal unmixing [70], [71] and deep learning-based unmixing approaches, particularly convolutions, autoencoders and transformer/attention architectures, to better integrate spectral–spatial information and model spectral variability [72], [73]. While these advances are crucial for achieving accurate and reliable separation, it is still imperative to develop computationally efficient algorithms suitable for distributing the computational load.

In this context, the proposed `DPGD` method seeks efficient and scalable processing of hyperspectral images, taking advantage of cloud computing and Spark parallel processing capabilities for unmixing tasks. Inspired by MDA and leveraging the advantages of PySpark, the proposed `DPGD` method is specifically designed to address the computational challenges imposed by the high spectral dimensionality of hyperspectral data. This is achieved through the partitioning of RDDs to efficiently distribute computational workload and minimize communication overhead. This approach significantly increases the efficiency when extracting endmembers and estimating their abundances from high-dimensional hyperspectral

Authorized licensed use limited to: Hohai University Library. Downloaded on November 09,2026 at 11:45:48 UTC from IEEE Xplore. Restrictions apply.

images. In Section III, we provide a detailed explanation of how the partitioning process is carried out, as well as the operational mechanisms and communication strategies among these partitions.

A. Local Endmember Extraction Using Geometric Distance

The input data are first partitioned for local endmember extraction. Given the hyperspectral image $\mathbf{X} \in \mathbb{R}^{N\times M}$, where N denotes the number of pixels in the scene and M is the number of spectral bands, each row represents a pixel vector $\mathbf{x}_i = \mathbf{X}[i,:] = \left[x_{i1}, x_{i2}, \ldots, x_{iM}\right]^\top \in \mathbb{R}^M$ which captures the spectral signature of the observed materials in all M spectral bands, with $i = 1, 2, \ldots, N$.

To enable efficient distributed processing, the proposed DPGD algorithm partitions the hyperspectral image $\mathbf{X}$ into P disjoint subsets $\mathbf{X}_1, \ldots, \mathbf{X}_P$, where each subset $\mathbf{X}_p \in \mathbb{R}^{N_p\times M}$ contains a portion of the original pixels. Here, $N_p = N/P$ ensures uniform partitioning across the P subsets

$$\bigcup_{p=1}^{P} \mathbf{X}_p = \mathbf{X}$$
$$\text{with} \quad \mathbf{X}_p = \left\{\mathbf{x}_i \middle| i \in \mathcal{I}_p\right\}, \quad \text{and} \quad \mathbf{X}_p \cap \mathbf{X}_j = \emptyset \quad \forall p \neq j \quad (8)$$

where $\mathcal{I}_p = \{i_1, i_2, \ldots, i_{N_p}\} \subset \{1, 2, \ldots, N\}$ is the set of indexes of the samples assigned to the pth partition. These partitions are then distributed across W nodes with storage and computing resources within the cluster to parallelize the computational workload, optimize resource utilization, and ensure correct computational load balancing. The number of partitions P is determined based on the dataset size and the cluster configuration to achieve an optimal tradeoff between efficiency and performance, whilst the number of nodes W is determined by the cluster configuration. Details are provided in Section III-B.

Within each node, the MDA is applied iteratively in each data partition, maximizing the geometric distances of the corresponding affine hull to extract K_{local} local endmembers, i.e., $\mathbf{S}_{\text{local}} = \{\mathbf{x}_A, \mathbf{x}_B, \mathbf{x}_C, \mathbf{x}_D, \ldots, \}$. For simplicity, these endmembers will be denoted as $\mathbf{S}_{\text{local}} = \{\mathbf{s}_i | i \in \mathcal{S}_p\}$, where $\mathcal{S}_p = \{A = i_1, B = i_2, \ldots, i_{K_{\text{local}}}\} \subset \{1, 2, \ldots, N_p\}$ is the subset of indexes of the endmember candidates extracted from the pth partition. Consequently, a total of PK_{local} candidates are obtained. This set of local candidates undergoes a final refinement step through a global MDA application, which consolidates and selects the final K_{global} endmembers.

Focusing on the pth partition, the MDA algorithm should proceed as follows.

1) The process would select the first endmember $\mathbf{x}_A$ with the greatest Euclidean distance from the origin, according to (3), i.e., $\mathbf{x}_A = \arg\max_{\mathbf{x}_i\in\mathbf{X}_p} \|\mathbf{x}_i\|$.
2) The second endmember $\mathbf{x}_B$ would be then selected to maximize the distance from $\mathbf{x}_A$, according also to (3), i.e., $\mathbf{x}_B = \arg\max_{\mathbf{x}_i\in\mathbf{X}_p} \|\mathbf{x}_i - \mathbf{x}_A\|_2$.
3) The third endmember $\mathbf{x}_C$ would be obtained to maximize the distance from the line formed by $\mathbf{x}_A$ and $\mathbf{x}_B$, following the distance defined by (4), i.e., $\mathbf{x}_C = \arg\max_{\mathbf{x}_i\in\mathbf{X}_p} d_{i_L_{\{AB\}}}$.
4) Once the plane defined by the first three endmembers is obtained, the fourth endmember $\mathbf{x}_D$ would be calculated by maximizing its distance to the plane as (5) indicates, i.e., $\mathbf{x}_D = \arg\max_{\mathbf{x}_i\in\mathbf{X}_p} d_{i_P_{\{ABC\}}}$.
5) Finally, candidates from the fifth to the K_{local}th would be extracted by maximizing their distance to the affine hull, following (6).

1) Endmember Initialization Through Random Sampling: Although partitioning the data results in $N \gg N_p$, which is a significant reduction in computational complexity, the associated computations are still expensive. To address this issue, the proposed DPGD algorithm begins by initializing $\mathbf{S}_{\text{local}} = []$ and $\mathcal{S}_p = \emptyset$ to empty. Also, it randomly selects three pixel vectors, $\mathbf{c}_1$, $\mathbf{c}_2$, and $\mathbf{c}_3$, from $\mathbf{X}_p$. These vectors define the initial candidate set

$$\mathbf{S}_{\text{init}} = \left\{\mathbf{c}_i \middle| i \in \mathcal{S}_{\text{aux}}\right\} \quad (9)$$

where

$$\mathcal{S}_{\text{aux}} = \{i_1, i_2, i_3\} \subset \left\{1, 2, \ldots, N_p\right\}. \quad (10)$$

These selected pixel vectors span an initial affine hull (indeed, an affine plane) denoted as $\text{Aff}\{\mathbf{c}_1\mathbf{c}_2\mathbf{c}_3\}$, from which endmembers $\mathbf{x}_A$, $\mathbf{x}_B$, and $\mathbf{x}_C$, are extracted, as illustrated in Fig. 1. It is observed that the procedure iteratively extracts the first endmembers by calculating distances using the residuals obtained after projecting each pixel vector $\mathbf{x}_i$ onto the affine hull at each iteration. Further details are provided in the following.

2) SVD-Based Normal Vector Estimation: As aforementioned in (9), matrix $\mathbf{S}_{\text{init}} = [\mathbf{c}_1, \mathbf{c}_2, \mathbf{c}_3]^T \in \mathbb{R}^{3\times M}$ is constructed as the initial set of randomly selected endmembers from $\mathbf{X}_p$. Next, the proposed DPGD algorithm computes the vector $\mathbf{w}$ orthogonal to $\text{Aff}(\mathbf{c}_1\mathbf{c}_2\mathbf{c}_3)$. This is achieved by computing the null space of $\mathbf{S}_{\text{init}}$, which consists of the set the set of all vectors $\mathbf{z} \in \mathbb{R}^M$ that satisfy

$$\mathbf{S}_{\text{init}}\mathbf{z} = \mathbf{0}. \quad (11)$$

It should be noted that $\mathbf{S}_{\text{init}}$ defines four fundamental subspaces related to its column and row spaces, namely, the column rank, row rank, and their respective null spaces. Thus, the DPGD relies on the singular value decomposition (SVD) to determine the null space of $\mathbf{S}_{\text{init}}$ with respect to its row space. This represents those directions orthogonal to $\text{Aff}\{\mathbf{c}_1\mathbf{c}_2\mathbf{c}_3\}$

$$\mathbf{S}_{\text{init}} = \mathbf{U}_p\,\mathbf{\Sigma}_p\,\mathbf{V}_p^\top \quad (12)$$

where $\mathbf{U}_p \in \mathbb{R}^{3\times 3}$ is an orthogonal matrix containing the left singular vectors, which span the row space $\mathbb{R}^3$; $\mathbf{\Sigma}_p \in \mathbb{R}^{3\times M}$ is a diagonal positive matrix containing the singular values; and $\mathbf{V}_p \in \mathbb{R}^{M\times M}$ is an orthogonal matrix containing the right singular vectors, which span the column space $\mathbb{R}^M$. In this context, the right singular vectors represent linear combinations of the columns of $\mathbf{S}_{\text{init}}$ and capture its geometric structure.

3) Computation of the Affine Hull Null Space: The null space of $\mathbf{S}_{\text{init}}$ consists of the subspace of $\mathbb{R}^M$ spanned by the right singular vectors associated with zero (or near-zero) singular values. These vectors do not contribute information on the structure of $\mathbf{S}_{\text{init}}$. As a result, those $\mathbf{v}_i \in \mathbf{V}_p$ corresponding

Authorized licensed use limited to: Hohai University Library. Downloaded on November 09,2026 at 11:45:48 UTC from IEEE Xplore. Restrictions apply.

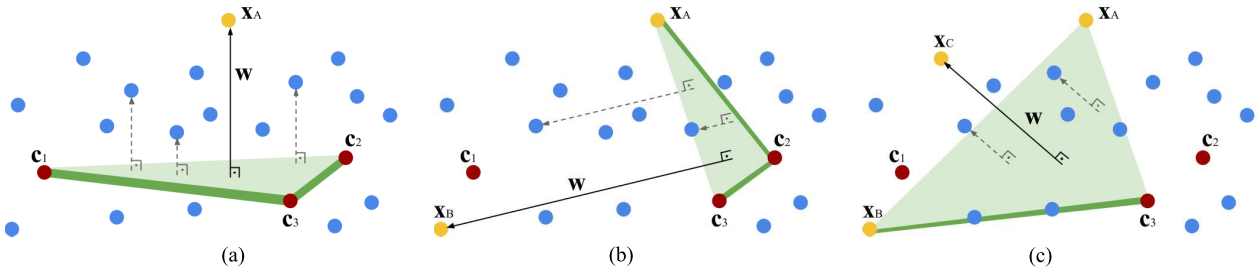


Fig. 1. Acquisition of the first three endmembers. The first step initializes the structures $\mathbf{S}_{\text{local}} = []$, $\mathcal{S}_p = \emptyset$, $\mathbf{S}_{\text{init}} = [\mathbf{c}_i | i \in \mathcal{S}_{\text{aux}}]$, and $\mathcal{S}_{\text{aux}} = \{i_1, i_2, i_3\} \subset \{1, \ldots, N_p\}$ and obtains the first endmember $\mathbf{x}_A$ by maximizing its distance to the initial affine hull Aff$\{\mathbf{c}_1\mathbf{c}_2\mathbf{c}_3\}$. Then, $\mathbf{S}_{\text{local}} = [\mathbf{x}_A]^T$ and $\mathcal{S}_p = \{A\}$ are updated. The second step updates the affine hull Aff$\{\mathbf{c}_2\mathbf{c}_3\mathbf{x}_A\}$ and finds the second endmember $\mathbf{x}_B$. Then, $\mathbf{S}_{\text{local}} = [\mathbf{x}_A, \mathbf{x}_B]^T$ and $\mathcal{S}_p = \{A, B\}$ are updated. The third step updates Aff$\{\mathbf{c}_3\mathbf{x}_A\mathbf{x}_B\}$ and finds the third endmember $\mathbf{x}_C$. Then, $\mathbf{S}_{\text{local}} = [\mathbf{x}_A, \mathbf{x}_B, \mathbf{x}_C]^T$ and $\mathcal{S}_p = \{A, B, C\}$ are updated. Thereafter, the selected endmembers are no longer removed and are all used to calculate the next candidates. (a) First step, (b) second step, and (c) third step.

to a zero singular value form a basis for the null space of $\mathbf{S}_{\text{init}}$. The dimension of this null space, i.e., the number of linearly independent basis vectors, is obtained via the rank-nullity theorem, which states that

$$\text{rank}\,(\mathbf{S}_{\text{init}}) + \dim\,(\mathcal{N}\,(\mathbf{S}_{\text{init}})) = M \tag{13}$$

where $\mathcal{N}(\cdot)$ denotes the null space. As $\mathbf{S}_{\text{init}}$ contains three rows, the rank is at most $\min(3, M)$. Furthermore, by reformulating (13) and assuming full row rank, it can be interpreted that $\dim(\mathcal{N}(\mathbf{S}_{\text{init}})) = M - 3$. This implies that there exist $M - 3$ independent vectors that span the null space. Finally, the normal vector $\mathbf{w}$ of the affine hull is computed as the average of the obtained $M - 3$ null space vectors

$$\mathbf{w} = \frac{1}{M-3} \sum_{i=1}^{M-3} \mathbf{v}_i. \tag{14}$$

Since each $\mathbf{v}_i$ belongs to the null space of $\mathbf{S}_{\text{init}}$, (11) is reformulated as

$$\mathbf{S}_{\text{init}}\mathbf{v}_i = \mathbf{0} \quad \forall i = 1, \ldots, M-3. \tag{15}$$

Thus, by linearity

$$\mathbf{S}_{\text{init}}\mathbf{w} = \mathbf{0}. \tag{16}$$

4) Geometric Distance Evaluation for Endmember Selection: Once $\mathbf{w}$ is obtained, the distance from each pixel vector $\mathbf{x}_i \notin \mathbf{S}_{\text{init}}$ to Aff$\{\mathbf{c}_1\mathbf{c}_2\mathbf{c}_3\}$ is calculated as

$$d_i = \frac{\left|\mathbf{w}^\top (\mathbf{x}_i - \mathbf{c}_1)\right|}{\|\mathbf{w}\|} \tag{17}$$

where $\mathbf{c}_1$ represents a reference point within the original affine hull.

Finally, the pixel vector $\mathbf{x}_i$ with maximum distance $d_{i_\text{Aff}\{\mathbf{c}_1\mathbf{c}_2\mathbf{c}_3\}}$ exceeding a threshold δ is selected as the first endmember $\mathbf{x}_A$

$$\begin{aligned} \mathbf{x}_A = \arg\max_{\mathbf{x}_i \in \mathbf{X}_p} \quad & d_{i_\text{Aff}\{\mathbf{c}_1\mathbf{c}_2\mathbf{c}_3\}} \\ \text{s.t.} \quad & d_{i_\text{Aff}\{\mathbf{c}_1\mathbf{c}_2\mathbf{c}_3\}} > \delta. \end{aligned} \tag{18}$$

5) Affine Hull Update and Iteration: Knowing the first endmember $\mathbf{x}_A = \mathbf{s}_1$, it becomes part of the set of K_{local} endmembers, updating the matrix of local endmembers $\mathbf{S}_{\text{local}} = [\mathbf{x}_A]^T$, and storing its index into the list $\mathcal{S}_p = \{A\}$. The auxiliary structures are also updated as $\mathbf{S}_{\text{init}} = [\mathbf{c}_2, \mathbf{c}_3, \mathbf{x}_A]^T$ and $\mathcal{S}_{\text{aux}} = \{i_2, i_3, A\}$, removing the first random candidate and introducing the new endmember.

The algorithm repeats steps 1)–5) to extract $\mathbf{x}_B$ and $\mathbf{x}_C$, removing the information of the initial random candidates for that of the final endmembers. Therefore, at the end of the third iteration, the data structures store the following information: $\mathbf{S}_{\text{local}} = \mathbf{S}_{\text{init}} = [\mathbf{x}_A, \mathbf{x}_B, \mathbf{x}_C]^T$ and $\mathcal{S}_p = \mathcal{S}_{\text{aux}} = \{A, B, C\}$. Thereafter, these structures store the final endmembers calculated by the p partition.

For the fourth and subsequent endmembers, the procedure follows the same steps.

1) Initiate $\mathbf{S}_{\text{init}} = \{\mathbf{s}_i \mid i \in \mathcal{S}_{\text{aux}}\}$ with indexes $\mathcal{S}_{\text{aux}} = \{i_1, \ldots, i_k\} \subset \{1, 2, \ldots, N_p\}$ and $k = 1, \ldots, K_{\text{local}}$.
2) Conduct SVD as (12) with $\mathbf{U}_p \in \mathbb{R}^{k \times k}$, $\mathbf{\Sigma}_p \in \mathbb{R}^{k \times M}$, and $\mathbf{V}_p \in \mathbb{R}^{M \times M}$.
3) Obtain the null space dimension, $\dim(\mathcal{N}(\mathbf{S}_{\text{init}})) = M - k$ and calculate $\mathbf{w} = (1/(M-k)) \sum_i \mathbf{v}_i$.
4) Compute the distances of each pixel to the affine hull Aff$\{ABCD\ldots\}$ and extract the new endmember, $\mathbf{s}_{k+1} = \arg\max_{\mathbf{x}_i \in \mathbf{X}_p} d_{i_\text{Aff}\{ABCD\ldots\}}$.
5) Update $\mathbf{S}_{\text{local}} = \mathbf{S}_{\text{init}}[\mathbf{s}_1, \ldots, \mathbf{s}_k, \mathbf{s}_{k+1}]^T$ and $\mathcal{S}_p = \mathcal{S}_{\text{aux}} = \{i_1, \ldots, i_k, i_{k+1}\}$.

This iterative process continues until the desired number of endmembers K_{local} is found or the maximum distance falls below δ, indicating that the affine hull represents the data within the partition.

6) Global Endmember Refinement: The total PK_{local} local endmembers from all partitions are collected and a *reduction* is performed to obtain the desired K_{global} endmembers. Indeed, a nondistributed SVD on the aggregated endmember matrix $\mathbf{S}_{\text{global}}$ is conducted

$$\mathbf{S}_{\text{global}} = \begin{bmatrix} \mathbf{S}_1 \\ \mathbf{S}_2 \\ \vdots \\ \mathbf{S}_P \end{bmatrix} \in \mathbb{R}^{(\text{PK}_{\text{local}}) \times M} \tag{19a}$$

$$\mathbf{S}_{\text{global}} = \mathbf{U}_{\text{global}} \mathbf{\Sigma}_{\text{global}} \mathbf{V}_{\text{global}}^\top. \tag{19b}$$

The global SVD in (19b) refines the local endmember set by capturing variability across all partitions. The final set of endmembers is obtained from $\mathbf{S}_{\text{global}}$, and the fully constrained least-squares (FCLS) function is used to combine the extracted endmembers and hyperspectral images to estimate

Authorized licensed use limited to: Hohai University Library. Downloaded on November 09,2026 at 11:45:48 UTC from IEEE Xplore. Restrictions apply.

Algorithm 1 Distributed Endmember Extraction

Require: Hyperspectral data $\mathbf{X} \in \mathbb{R}^{N \times M}$ stored in S3
Ensure: Extracted endmembers $\mathbf{S}_{\text{global}} \in \mathbb{R}^{K_{\text{global}} \times M}$

Data Loading:
Load $\mathbf{X} \in \mathbb{R}^{N \times M}$ into a DataFrame and convert to RDD
Split the RDD into P partitions
for each partition p in parallel **do**
  Local Processing:
  Load partition data $\mathbf{X}_p \in \mathbb{R}^{N_p \times M}$
  Initialize random indices $\mathcal{S}_{\text{aux}} = \text{rand_permut}(N_p)[:3]$
  Initialize local endmembers indices $\mathcal{S}_p = \emptyset$
  for $k = 1$ to K_{local} **do**
    Update candidate set $\mathcal{S}_{\text{aux}} = \text{unique}(\mathcal{S}_{\text{aux}} \cup \mathcal{S}_p)$
    if $\text{lenght}(\mathcal{S}_p) < 3$ **then**
      Select the last 3 endmembers from $\mathcal{S}_{\text{aux}}$, $\mathbf{S}_{\text{init}} = \mathbf{X}_p[\mathcal{S}_{\text{aux}}[-3], :]$
    else
      Use existing endmembers $\mathcal{S}_p$, $\mathbf{S}_{\text{init}} = \mathbf{X}_p[\mathcal{S}_{\text{p}}, :]$
    end if
    Compute SVD $\mathbf{S}_{\text{init}} \leftarrow$ Eq. (12)
    Determine $\dim(\mathcal{N}(\mathbf{S}_{\text{init}})) \leftarrow$ Eq. (13)
    Compute mean normal vector $\mathbf{w} \leftarrow$ Eq. (14)
    Calculate distances $\mathbf{d}_{\text{Aff}\{\text{ABCD...}\}} \leftarrow \frac{|\mathbf{X}_p \mathbf{w}|}{\|\mathbf{w}\|}$
    if $\max(\mathbf{d}_{\text{Aff}\{\text{ABCD...}\}}) > \delta$ **then**
      Update $\mathcal{S}_p = \mathcal{S}_p \cup \arg\max(\mathbf{d}_{\text{Aff}\{\text{ABCD...}\}})$
    else
      break
    end if
  end for
  Update $\mathbf{S}_{\text{local}} = \mathbf{X}_p[\mathcal{S}_{\text{p}}, :]$
  Emit local endmembers $\mathbf{S}_{\text{local}}$
end for
Global Refinement:
Collect all local endmembers into $\mathbf{S}_{\text{global}}$
Refine endmembers on $\mathbf{S}_{\text{global}} \leftarrow$ Eq. (19)
Output: Final endmembers $\mathbf{S}_{\text{global}}$

the respective abundance maps. Algorithm 1 provides the pseudocode of the procedure.

B. Cloud-Based Parallel Execution Using Spark

To fully exploit the potential of cloud computing for big data processing and effectively apply it to hyperspectral unmixing, Apache Spark has been adopted as the distributed processing engine, and Amazon Web Services (AWS) as the cloud computing infrastructure. This selection was made after careful consideration of algorithmic needs and data access patterns, taking into account the performance and scalability requirements.

Apache Spark was selected as the computational backbone for the proposed DPGD method due to its advanced capabilities in in-memory data processing, lazy evaluation, and support for iterative algorithmic patterns. These features are critical for efficiently executing the repeated geometric distance computations intrinsic to DPGD. In contrast to traditional batch-oriented paradigms such as Hadoop MapReduce, Spark's RDD abstraction enables the reuse of intermediate computation results, thereby minimizing disk I/O and enhancing computational performance. Although alternative frameworks such as Dask and Apache Flink provide comparable functionalities, Spark's architectural maturity, proven fault tolerance, and seamless integration with resource managers like YARN made it the most suitable choice for addressing the deployment and scalability requirements of the experimental pipeline.

In parallel, the AWS cloud platform was selected as the deployment infrastructure due to its scalability, reliability, and comprehensive integration with distributed data processing frameworks. First, Amazon Simple Storage Service (S3) offers a highly durable and cost-effective storage solution, specifically designed for high-volume data management in distributed computing environments. Its native compatibility with Apache Spark via the S3A connector enables efficient parallel data loading and minimizes I/O bottlenecks across executors. Second, AWS Elastic MapReduce (EMR) provides a fully managed and preconfigured Apache Spark environment that supports dynamic resource allocation, robust fault tolerance through YARN, and automatic cluster scaling. These features align well with the computational demands of the DPGD algorithm. Third, the research team's prior expertise with the AWS ecosystem significantly streamlined the development process, enabling rapid prototyping, seamless deployment, and reliable system integration. An overview of the integrated computational pipeline and its constituent components is illustrated in Fig. 2.

While alternative cloud service providers, such as Microsoft Azure and Google Cloud Platform, offer comparable solutions (e.g., Azure HDInsight and GCP Dataproc), the selection of AWS was the result of a thorough evaluation of performance, scalability, deployment flexibility, and compatibility with the existing development environment and workflow. Accordingly, the use of Apache Spark on AWS EMR was not a discretionary choice but a deliberate and empirically justified decision aimed at enhancing implementation efficiency and ensuring reproducibility of the experimental pipeline.

1) Apache Spark for Gigabyte-Scale Data Analysis: As pointed out before, the proposed `DPGD` adopts the flexible computational paradigm of Apache Spark [74], which provides in-memory processing and optimized iterative execution. This significantly improves the efficiency of gigabyte-scale distributed data analysis in fields such as hyperspectral imaging.

From an architectural perspective, Spark defines three runtime components. First, Spark organizes any application in a manager–worker scheme by defining independent sets of processes, i.e., one *Driver Program* and multiple *Executor Processes*. In contrast to the rigid map-reduce, the driver translates the application into a directed acyclic graph (DAG) to represent and optimize the operation flow and decomposes it into stages of tasks, coordinating their execution in the executors. It also manages metadata, collects and processes executors' data, and interacts with the cluster manager to request and manage resources. On the other hand, each executor runs its tasks on its allocated computing resources while managing memory and cores for optimized distributed

Authorized licensed use limited to: Hohai University Library. Downloaded on November 09,2026 at 11:45:48 UTC from IEEE Xplore. Restrictions apply.

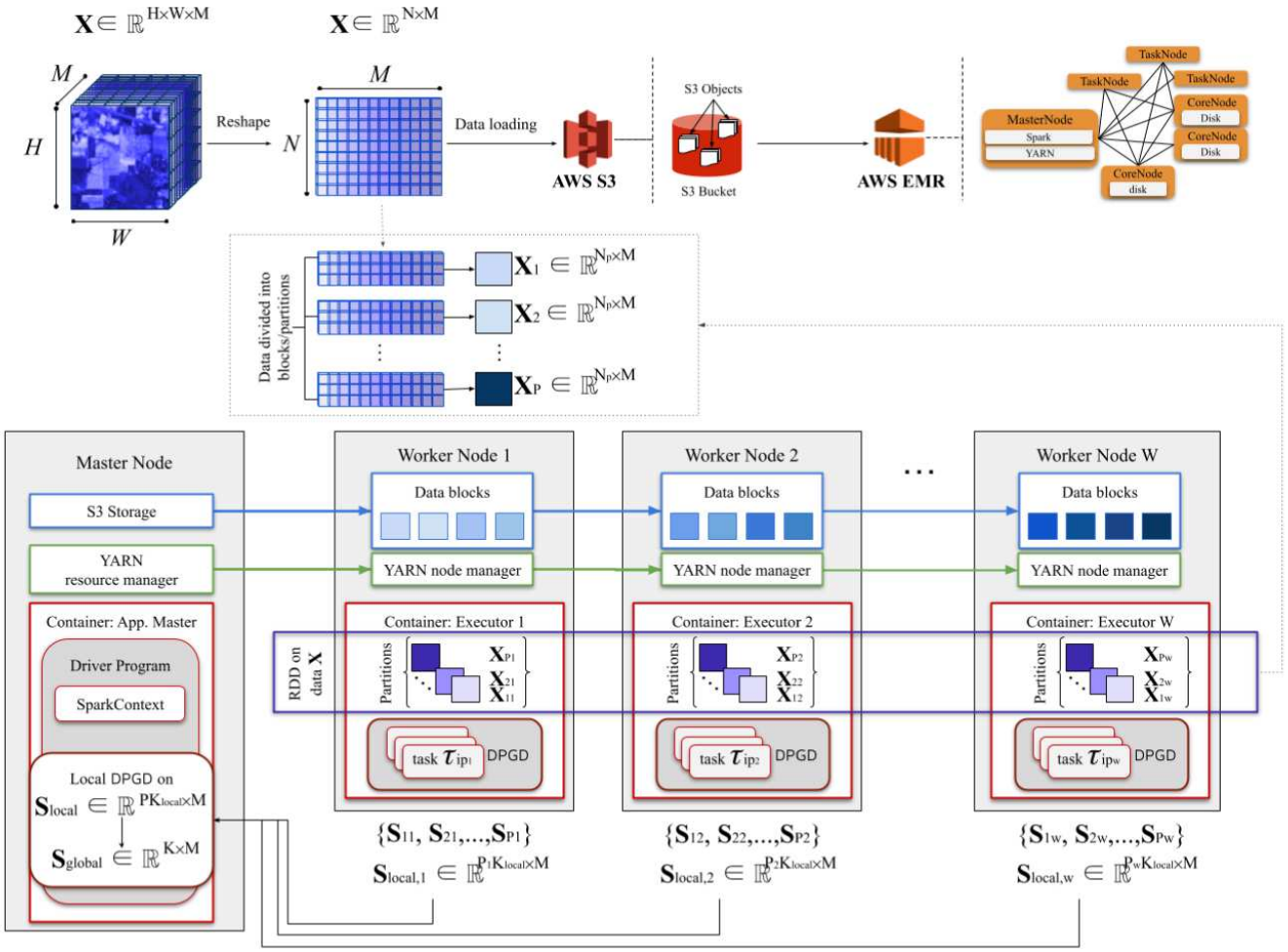


Fig. 2. Workflow of the proposed DPGD method for hyperspectral image unmixing. Notation has been adapted to include the number of executor that obtains the K_{local} candidates on its assigned data partitions. Focusing on the upper row, from left to right, the HSI data are reshaped into a 2-D array and stored in the AWS S3 bucket as a set of data blocks. These are distributed within the AWS EMR cluster, with Spark and YARN coordinating the storage and data transfer to the worker nodes. The bottom row details the distribution procedure of the DPGD. The driver program, running inside the application manager on the manager node, initializes the SparkContext, coordinates the execution plan, and manages the distribution of tasks to the executors on the worker nodes. Each executor processes its assigned data from the RDD in parallel, and the partial results are finally aggregated to produce the output of the DPGD algorithm.

computation. Furthermore, Spark relies on a *pluggable cluster manager* to acquire resources, with common options, including Spark Standalone, Hadoop YARN, Kubernetes, and Mesos. For efficient deployment of the proposed DPGD, YARN has been selected as the resource manager. This enables Spark to dynamically allocate and deallocate resources according to workload demands. Furthermore, Cluster Mode deployment is used for better resilience. In this setup, the driver runs inside a dynamically allocated Manager container within the YARN cluster, while executors are distributed across multiple worker nodes, with the cluster manager overseeing all application-related processes.

To provide increased flexibility, Spark not only introduces the DAG in place of the rigid MapReduce model but also replaces traditional Hadoop's disk-based storage model with the RDD. The RDD represents a distributed collection of data and, unlike traditional Hadoop data structures, it enables in-memory computation, supports lazy evaluation to optimize execution, and provides fault tolerance through lineage tracking, ensuring efficient and reliable distributed data processing. The only-read RDD is loaded from and stored in Amazon S3. Also, it is divided into partitions that can be processed in parallel.

In DPGD, image $\mathbf{X}$ is loaded into an RDD from S3 and partitioned into P partitions, as in (8) and Algorithm 1. Operations on the RDD involve L lazy *transformations* $\mathcal{T} = \{T_1, \ldots, T_L\}$—such as map, filter, and union—and $\tilde{L}$ *actions* $\mathcal{A} = \{A_1, \ldots, A_{\tilde{L}}\}$—such as collect, count, and reduce—Each transformation T_i maps an input RDD to an output RDD, processing each $\mathbf{X}_p$ independently. In contrast, each action A_j triggers the execution of all preceding transformations in its lineage $\mathcal{T}$, returning results or writing data to external storage

$$T_i : \text{RDD}_{\text{in}} \longrightarrow \text{RDD}_{\text{out}}, \text{ with } T_i(\mathbf{X}) = \bigcup_{p=1}^{P} T_i(\mathbf{X}_p) \quad (20a)$$

$$A_j : \text{RDD}_{\text{some}} \longrightarrow R \cup S, \text{ with } A_j(\mathbf{X}) = f\left(\bigcup_{p=1}^{P} \mathbf{X}_p\right) \quad (20b)$$

Authorized licensed use limited to: Hohai University Library. Downloaded on November 09,2026 at 11:45:48 UTC from IEEE Xplore. Restrictions apply.

where R and S indicate the delivery of a result, either to the driver or to external storage, and $f(\cdot)$ is the aggregation or collection of data across partitions. Taken together, $\mathcal{T}$ and $\mathcal{A}$ define the DAG $\mathcal{G} = (V, E)$, where each node in V is a transformation or an action, and there is an edge $(v_i \rightarrow v_j) \in E$ that connects the output RDD of node v_i as the input RDD of node v_j.

The driver process, which includes the *SparkContext*, is responsible for constructing and managing the DAG $\mathcal{G}$. It records the sequence of transformations $\mathcal{T}$ and actions $\mathcal{A}$, and maintains dependencies between RDDs. As a result, the driver defines the function $\delta : \mathcal{T} \cup \mathcal{A} \rightarrow \mathcal{G}$, which maps each transformation or action to a node in the DAG $\mathcal{G}$. This map establishes connections between nodes based on RDD dependencies and ensures proper execution flow. Furthermore, the SparkContext registers these transformations and actions in the logical execution plan

$$\text{Plan}(\mathcal{G}) = A_j\left(T_i \circ T_{i-1} \circ \cdots \circ T_1\left(\text{RDD}_{\text{in}}\right)\right) \quad \forall j \in \left[1, \tilde{L}\right] \text{ and } \quad \forall i \in [1, L]. \tag{21}$$

This physical plan defines a pipeline that remains lazily evaluated until an action is invoked. When an action is triggered, the SparkContext submits the DAG to the scheduler, breaking it into several stages. Each stage groups those transformations that can be performed without shuffles or wide dependencies. At each stage, a set of tasks is spawned and dispatched to executors for distributed execution in $\{\mathbf{X}_1, \ldots, \mathbf{X}_P\}$.

To ensure efficient task execution across the distributed system, the cluster manager dynamically allocates computing resources among a set of physical or virtual *worker nodes*, each characterized by a CPU capacity of C_w and a memory capacity of $\mathcal{M}_w$, where w indicates the wth node $\forall w \in [1, W]$. This ensures that execution environments are properly provisioned. Knowing the total number of worker nodes W, the driver submits a resource request to the cluster manager for a number of executors E or for a total number of CPU cores throughout the cluster, and the manager defines the resource-allocation function μ : {Executors} $\longrightarrow$ {Worker Nodes}, which place each executor on some worker node, subject to $\sum C_w \leq \sum$(total number of CPUs). This means that the number of executors is set according to the number of cores of the worker. In the case of the DPGD, one executor per node is set, i.e., $W = E$.

Once assigned to the worker nodes, the executors operate as independent processing units, performing their assigned computations on data partitions. In this context, each executor is an independent process that runs on a worker node with access to a portion of the CPU and memory to carry out its tasks, such as transformations on the RDDs. Executors can also cache intermediate results, including partial RDDs, or store data in local storage. Denoting the wth executor as e_w and its set of assigned partitions as $\{\mathbf{X}_{1_w}, \ldots, \mathbf{X}_{P_w}\} \subset \{\mathbf{X}_1, \ldots, \mathbf{X}_P\}$, it receives from the driver a task $\tau_{ip_w} \equiv (T_i, \mathbf{X}_{p_w})$ to apply the transformation $T_i \in \mathcal{T}$ on its pth assigned partition, $\forall p_w \in [1, P]$. If possible, the executor conducts T_i in memory, producing a new RDD partition as in (20a)

$$\tilde{\mathbf{X}}_{p_w} = T_i\left(\mathbf{X}_{p_w}\right). \tag{22}$$

If subsequent transformations require the new RDD, the executor retains it in memory.

Since Spark processes data through a series of transformations structured as a DAG, cached partitions are crucial for efficient execution and scheduling. Specifically, the DAG comprises G stages $\mathcal{G} = \{\mathcal{G}_1, \ldots, \mathcal{G}_G\}$, where each one groups transformations with narrow dependencies that execute without data shuffling. Each $\mathcal{G}_g$ is divided into T_g tasks to run in all partitions, $\mathcal{G}_g = \{\tau_{1_g 1}, \ldots, \tau_{1_g P}, \ldots, \tau_{T_g 1}, \ldots, \tau_{T_g P}\}$. This ensures full parallel processing of the dataset. Furthermore, the driver maintains the topological order over the stages, $\mathcal{G}_1 \prec \mathcal{G}2 \prec \cdots \prec \mathcal{G}_g$, ensuring that stage $g + 1$ begins its execution only after all tasks in stage g are completed across all partitions. This strict order prevents partial results from incomplete transformations. Moreover, in the event that an executor fails during $\tau_{i_g p_w}$, Spark consults the lineage graph to determine which transformations produced $\tilde{\mathbf{X}}_{p_w}$. Since RDDs are immutable and transformations are pure functions, Spark can efficiently recompute lost partitions on any healthy executor, without additional overhead, ensuring fault tolerance and consistency in execution.

2) Integration With AWS: To efficiently process high-dimensional data, AWS S3 and EMR have been used for persistent storage and computation, respectively.

The data storage and distribution mechanism adopted by the DPGD framework is based on Amazon S3,[1] a storage-as-a-service (SaaS) infrastructure that provides highly durable, scalable, and secure object-based cloud storage. Amazon S3 guarantees an unparalleled percentage of data durability by redundantly storing objects across multiple geographically separated Availability Zones within an AWS Region. Its design supports high availability, low-latency access, and a pay-per-use billing model, making it particularly well-suited for data-intensive and dynamically scaling scientific workflows.

To efficiently distribute hyperspectral data, the S3 Standard storage class has been selected. This class is optimized for frequently accessed data, offering millisecond-scale latency and high throughput, which are essential for efficient execution of parallel processing tasks in hyperspectral unmixing. In this context, and from an architectural perspective, Amazon S3 uses two types of resources, i.e., *buckets* and *objects*, to manage large volumes of data (as depicted in Fig. 2). Specifically, S3 uses a nonhierarchical flat namespace model, wherein user data are encapsulated together with its metadata as a set of discrete and immutable units called objects. Each object may contain two components: the binary payload, which represents the actual data content (e.g., a hyperspectral image file), and an optional, customizable set of metadata. Objects are uniquely identified by a key, and are stored within a logical container known as bucket. The combination of the bucket name and the object key forms a globally unique identifier for each object. This architecture overcomes the scalability limitations

[1] https://aws.amazon.com/s3

Authorized licensed use limited to: Hohai University Library. Downloaded on November 09,2026 at 11:45:48 UTC from IEEE Xplore. Restrictions apply.

of traditional fixed-block storage systems by decoupling the data and metadata layers, thereby enabling efficient horizontal scaling and flexible data organization. Indeed, objects can be accessed via a RESTful HTTP-based API or natively through distributed data processing frameworks such as Apache Spark, using the Hadoop-compatible S3A filesystem interface.

In this sense, the S3A connector allows Spark to interact with S3 as if it were a native file system, supporting seamless and scalable input/output operations across large distributed workloads. The S3A connector enables secure authentication through AWS identity and access management (IAM), translates filesystem operations into atomic HTTPS requests, and enhances data transfer performance using parallelized multipart upload and download mechanisms. In the context of hyperspectral datasets, the raw hyperspectral image is stored in an S3 bucket, and Apache Spark performs dynamic data partitioning through a two-phase process: 1) *logical decomposition* and 2) *distributed data retrieval*.

Regarding the first step, Spark (through its Spark driver) interacts with the S3 metadata layer to enumerate the objects associated with a dataset. This process provides the driver with the object keys and metadata (e.g., file size and structure) to construct a logical representation of the dataset. In particular, the hyperspectral scene is stored as a CSV. Then, to enable fine-grained parallel unmixing, the logical decomposition phase divides the dataset at the pixel level, ensuring that each pixel (comprising its full spectral vector) can be independently assigned to a Spark task for subsequent analysis. Resulting partitions are derived using byte-range approximations, computed as

$$\text{Partition Count} = \left\lceil \frac{\text{Dataset Size(MB)}}{\text{Block Size(MB)}} \right\rceil \tag{23}$$

where the block size is configurable (128 MB by default). This ensures load balancing.

Regarding the second step, Spark executors autonomously retrieve their assigned data partitions directly from Amazon S3, establishing multiple parallel connections without requiring mediation by the Spark driver. This decentralized data access pattern effectively mitigates potential I/O bottlenecks and promotes full utilization of the cluster aggregate bandwidth. Each executor independently resolves its specific partition boundaries, which are defined in terms of byte offsets for noncolumnar formats. This takes advantage of the low-latency S3 provided by the global namespace to maximize concurrent data access. As a result, linear scalability is ensured, as network bandwidth and request throughput grow with cluster size.

By decoupling storage from computations, S3 enables the DPGD framework to process gigabyte-scale hyperspectral datasets without on-premises storage limitations.

Once the raw hyperspectral image is stored in an S3 bucket, the proposed DPGD is then deployed on the EMR cluster and runs under the default Apache Spark configuration in the YARN cluster mode. The EMR cluster defines three types of instances, known as: 1) *manager node*, which coordinates the cluster and allocates the YARN ResourceManager and the Spark driver; 2) *core nodes*, which run the YARN NodeManager and Spark executors and have capacity to store data; and 3) *task nodes*, which also run the YARN NodeManager and Spark executors. The deployment offers high flexibility, regarding the instance allocation, and fault tolerance in the management of compute resources. Specifically, YARN dynamically allocates and manages computational resources across the cluster and, by default, supports automatic task reassignment in the event of node failures, thus enhancing the robustness and reliability of the distributed environment. However, no task failures were observed during the execution of the experiments, and therefore, the reported execution times reflect fault-free execution. Evaluating the impact of failures and explicitly validating fault tolerance mechanisms under controlled disruption conditions remains an important direction for future work. As a result, Fig. 2 focuses mainly on worker nodes.

When the Spark job is submitted, the YARN Resource Manager launches the application manager, which in turn requests the required number of containers to host the Spark executors. These executors are deployed across the EMR worker nodes. EMR also offers flexibility in organizing physical/virtual resources, handling the placement of core/task nodes, and executor containers transparently. Indeed, EMR conducts a dynamic allocation by default, allowing executors to be dynamically added or removed in response to workload fluctuations. In the DPGD algorithm, the co-location of Core/Taks nodes, YARN NodeManagers, and Spark executor containers is prioritized whenever possible. This co-location strategy enhances data locality, enabling computation to occur closer to where the data reside and thereby minimizing network overhead. As a result, one executor is allocated per worker node.

Once the executors are provisioned, data processing begins when the SparkContext reads the data from the S3 path and loads it into memory as an RDD. This enables in-memory computation for improved performance. The RDD is logically divided into P partitions of 128 MB each, aligning with the default file system block size, as defined in (23), which also determines the number of partitions Spark will create for a given dataset. In a real-world scenario, a dataset of 36.3 GB results in approximately 290 partitions.

Following Algorithm 1, a local version of the DPGD method is iteratively run on each assigned partition of an executor to extract K_{local} local endmembers. Each iteration involves the following key computational steps, revised in Section III-A. After all executors have completed this local extraction, the driver collects in $\mathbf{S}_{\text{local}} \in \mathbb{R}^{\text{PK}_{\text{local}} \times M}$ and the subsets $\mathbf{S}_{\text{local},w} \in \mathbb{R}^{P_w K_{\text{local}} \times M}$ emitted by each executor. A final refinement merges and prunes these local candidates to derive the definitive global endmembers $S_{\text{global}} \in \mathbb{R}^{K_{\text{global}} \times M}$. It is worth noting that the distributed processing framework assigns the P partitions across W nodes in the cluster. Hence, on average, the node w is responsible for processing approximately $P_w = P/W$ partitions concurrently $\forall w \in [1, W]$. This allocation ensures that each node performs a corresponding number of optimized DPGD in parallel, thereby accelerating the overall extraction process.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 09,2026 at 11:45:48 UTC from IEEE Xplore. Restrictions apply.

IV. Experiments

A. Experimental Environment

To validate the scalability and practical feasibility of the proposed method, we conducted experiments using a cloud-based distributed computing environment specifically tailored for high-volume data analysis. As previously stated, the proposed DPGD method is deployed on an AWS EMR cluster, which cluster consists of m5.xlarge instances, each providing four vCPUs and 16 GiB of RAM. The input hyperspectral data are stored and transferred from AWS S3 using AWS's distributed transfer capabilities.

The setup is designed to simulate realistic large-scale unmixing scenarios and to facilitate the assessment of algorithm performance under different computational loads. All performance measurements reported in Section IV-E correspond to the mean and standard deviation over five independent executions for each experimental configuration (dataset size $\times$ number of nodes).

B. Dataset Descriptions

With the aim of evaluating the performance and scalability of the proposed DPGD, a comprehensive set of experiments was conducted using two well-known hyperspectral datasets, **Cuprite** and **Samson**. These datasets were selected to represent both small-scale, well-defined unmixing scenarios and larger, more complex real-world cases, thereby enabling the assessment of the method's scalability and robustness.

1) Cuprite Dataset: The Cuprite dataset is one of the most widely used hyperspectral datasets to assess the performance of endmember extraction algorithms. In the experiments, a subset of the scene was used, comprising 250 $\times$ 191 pixels with 224 spectral bands ranging from 0.4 to 2.5 μm and a nominal spectral resolution of 10 nm. To improve data quality, bands 1–6, 105–115, 150–170, and 222–224 were excluded due to water absorption and low SNR, resulting in a total of 188 spectral bands retained for analysis. The Cuprite dataset primarily contains twelve materials: Alunite, Andradite, Buddingtonite, Dumortierite, Kaolinite (two variants), Muscovite, Montmorillonite, Nontronite, Pyrope, Sphene, and Chalcedony.

2) Samson Dataset: The Samson dataset is a widely used hyperspectral image consisting of 952 $\times$ 952 pixels with 156 spectral channels. The scene covers wavelengths from 401 to 889 nm, with a spectral resolution of 3.13 nm. Due to the high computational cost of processing the full image, we select a smaller region of interest consisting of 95 $\times$ 95 pixels, starting from the (252, 332)th pixel of the original dataset. This selected region includes three primary materials: soil, tree, and water, which are commonly utilized in spectral unmixing tasks.

Given the limited size of this subset, we replicated the dataset several times to create a larger $5 \cdot 10^6 \times 156$ synthetic dataset. This approach preserves the spectral characteristics of the original image while increasing the data volume, facilitating scalability testing. The replication process ensures that DPGD can be effectively evaluated under high-volume data conditions.

Both images contribute different characteristics to the evaluation. The Samson dataset, replicated to simulate high-volume data, is used to assess the scalability of the proposed approach. In contrast, the Cuprite dataset provides a complex, real-world scenario, enabling the robustness of the method to be tested under diverse and challenging spectral conditions.

C. Evaluation Metrics

In this study, the spectral angle distance (SAD) is used to evaluate the performance of different unmixing methods for endmember extraction. The SAD is calculated as

$$\mathrm{SAD}\left(\mathbf{s}_n, \hat{\mathbf{s}}_n\right) = \arccos \frac{\mathbf{s}_n \cdot \hat{\mathbf{s}}_n}{\|\mathbf{s}_n\|_2 \, \|\hat{\mathbf{s}}_n\|_2} \tag{24}$$

where $\mathbf{s}_n$ and $\hat{\mathbf{s}}_n$ are real and estimated endmember signatures, respectively. Additionally, we use the root-mean-square error (RMSE) to assess the performance of different algorithms in abundance estimation, defined as

$$\mathrm{RMSE}\left(\hat{\mathbf{a}}_j, \mathbf{a}_j\right) = \sqrt{\frac{1}{\mathrm{Np}} \sum_{j=1}^{\mathrm{Np}} \left(\hat{\mathbf{a}}_j - \mathbf{a}_j\right)^2} \tag{25}$$

where $\mathbf{a}_j$ is the real abundance map and $\hat{\mathbf{a}}_i$ is the estimated abundance map.

D. Effectiveness Evaluation

1) Experiments on Cuprite Dataset: Table I provides the quantitative results of the endmember extraction procedure conducted by the **MDA** [18], the **NMDA** [19], and the proposed DPGD unmixing method in the Cuprite scene. Each method is evaluated based on the SAD values computed for the 12 specific mineral endmembers comprising Cuprite. Also, the Mean SAD, which averages the 12 values, is evaluated. It should be noted that lower SAD values indicate better estimation accuracy (i.e., the estimated endmember spectrum is closer in direction to the reference), whilst the Mean SAD summarizes the overall performance across all materials.

The results show that the MDA effectively extracts the endmembers of some minerals, such as pyrope, reaching the minimum SAD of 0.0599. Also, regarding dumortierite, the MDA provides an SAD of 0.0701, whilst in kaolinite-1 and alunite, it provides 0.0794 and 0.0859, respectively. However, the proposed DPGD is very close to these values; e.g., for pyrope, the proposed method provides the second-best result of 0.0657, showing an SAD only 0.0058 points higher. For dumortierite, the DPGD also achieves the second-best result, with 0.0319 points higher. Regarding kaolinite-1, the DPGD and the NMDA provide similar results, 0.0054 points worse than MDA. Finally, in alunite, the DPGD provides a similar result. For the remaining minerals, MDA usually provides the highest SAD (it is the third method in 5 minerals, compared to 3 in which it ranks second). Furthermore, the high SAD of MDA in Sphene (0.3445) inflates its Mean SAD. This suggests a higher sensitivity to specific mineral spectra.

Focusing on NMDA, it overcomes the weaknesses of MDA by normalizing domain shifts and achieves lower

Authorized licensed use limited to: Hohai University Library. Downloaded on November 09,2026 at 11:45:48 UTC from IEEE Xplore. Restrictions apply.

TABLE I
QUANTITATIVE RESULTS BY DIFFERENT UNMIXING METHODS FOR THE CUPRITE DATASET. THE BEST RESULTS ARE SHOWN IN BOLD

Methods	SAD												Mean SAD
	Alunite	Andradite	Buddingtonite	Dumortierite	Kaolinite 1	Kaolinite 2	Muscovite	Montmorillonite	Nontronite	Pyrope	Sphene	Chalcedony	
MDA [18]	**0.0859**	0.0903	0.1385	**0.0701**	**0.0794**	0.1013	0.0892	0.0622	0.0752	**0.0599**	0.3445	0.0753	0.1060
NMDA [19]	0.0966	**0.0681**	0.1159	0.1199	0.0848	**0.0627**	**0.0834**	0.0611	**0.0670**	0.1285	0.2209	**0.0701**	0.0983
DPGD	**0.0859**	0.0888	**0.0839**	0.1020	0.0848	0.0674	0.1239	**0.0565**	0.0755	0.0657	**0.0675**	0.0889	**0.0825**

SADs in a broader set of minerals. As a result, it provides the second-best Mean SAD value. In particular, NMDA offers superior results in kaolinite-2 (0.0627), nontronite (0.0670), andradite (0.0681), chalcedony (0.0701), and muscovite (0.0834). With the exception of muscovite (0.1239), nontronite (0.0755), and chalcedony (0.0889), where the proposed DPGD yields the highest SAD values—exceeding the best results by 0.0405, 0.0085, and 0.0188 points, respectively, the method achieves the second-best performance for kaolinite-2 (0.0674) and andradite (0.0888), with differences of 0.0047 and 0.0207 points above the optimal values. For the remaining minerals, although NMDA gives SAD values mostly below 0.1, indicating good estimation accuracy, it unexpectedly performs poorly on dumortierite and pyrope. In particular, its values for dumortierite and pyrope are 0.1199 and 0.1285.

Focusing on the proposed DPGD, the method demonstrates a remarkably consistent performance throughout the set of minerals. In particular, it achieves the lowest Mean SAD (0.0825), outperforming both MDA (0.1060) and NMDA (0.0983) in 0.0235 and 0.0158 points, respectively. This reflects its ability to provide stable and competitive estimates regardless of the mineral. In terms of individual results, DPGD provides the lowest SAD in three minerals, i.e., buddingtonite (0.0839), montmorillonite (0.0565), and sphene (0.0675), and shares the best result for alunite (0.0859) with MDA. Its performance in montmorillonite surpasses both MDA and NMDA, showing a substantial improvement of 0.0057 and 0.0046 points, respectively. Similarly, for sphene, DPGD achieves a significantly better SAD (0.0675) compared to MDA (0.3445) and NMDA (0.2209), clearly demonstrating its advantage in more challenging cases. As pointed out before, in the remaining minerals, DPGD consistently delivers the second-best SAD values in six cases: pyrope (0.0657), dumortierite (0.1020), kaolinite-2 (0.0674), andradite (0.0888), kaolinite-1 (0.0848), and muscovite (0.1239). In particular, the SADs for pyrope and kaolinite-2 are very close to the best-performing methods, with differences of only 0.0058 and 0.0047 points, respectively. These small margins reflect that DPGD remains competitive even where it is not the top performer. While DPGD does not yield the best results in nontronite (0.0755), chalcedony (0.0889), and muscovite (0.1239), the differences with the best-performing method are relatively minor in nontronite (0.0085) and chalcedony (0.0188). Only in muscovite does the gap increase to 0.0405, making it the least favorable case for DPGD, though still within a reasonable error range.

TABLE II
QUANTITATIVE RESULTS BY DIFFERENT UNMIXING METHODS FOR THE SAMSON DATASET. THE BEST RESULTS ARE SHOWN IN BOLD

Methods	SAD			Mean SAD	Mean RMSE
	Soil	Tree	Water		
MDA [18]	**0.0404**	**0.0219**	0.1304	0.0642	0.3150
NMDA [19]	0.0493	0.0407	**0.0788**	0.0563	0.2906
DPGD	0.0455	0.0284	0.0799	**0.0513**	**0.2602**

Overall, the DPGD method demonstrates greater generalizability in all minerals, avoiding extreme SAD values and delivering strong performance both in cases of easy and difficult unmixing. This consistency and resilience make it particularly attractive for practical hyperspectral unmixing applications, where balanced performance across varied endmembers is essential.

2) Experiments on Samson Dataset: Table II presents a comparison regarding the quantitative performance of the algorithms described in Section II, i.e., **MDA** [18] and **NMDA** [19], and the proposed DPGD in the Samson dataset, providing the SAD values when reconstructing soil, tree, and water endmembers, as well as the Mean SAD and RMSE metrics. In this sense, the lower the values, the better the performance.

The MDA method achieves excellent SAD values in two out of three materials, i.e., soil (0.0404) and tree (0.0219). These values indicate highly accurate spectral estimations for dry land and vegetation. However, its performance drops significantly in water, where the SAD value increases to 0.1304, which is nearly double that of NMDA and DPGD. This high error in water substantially raises the Mean SAD to 0.0642. Furthermore, MDA yields a mean RMSE of 0.3150, the highest among the three methods. This behavior is consistent with what was observed in the Cuprite dataset, i.e, the MDA can perform extremely well for specific materials but is significantly sensitive to others, leading to a lack of robustness in diverse scenes.

In contrast, the NMDA method presents a more balanced performance when processing all three endmembers. It provides the best SAD value for water (0.0788) and consistent results for soil (0.0493) and tree (0.0407). This stability across materials leads to a Mean SAD of 0.0563, better than MDA and relatively close to DPGD. Furthermore, NMDA achieves a lower Mean RMSE (0.2906) than MDA, highlighting an improvement in reconstruction accuracy. These results confirm that NMDA improves the generalization of

Authorized licensed use limited to: Hohai University Library. Downloaded on November 09,2026 at 11:45:48 UTC from IEEE Xplore. Restrictions apply.

TABLE III
MEAN EXECUTION TIME (MEAN ± SD) AND OBSERVED SPEEDUP FOR DIFFERENT DATASET SIZES AND NUMBER OF NODES

Dataset (GB)	Nodes	Time	Speedup	Dataset (GB)	Nodes	Time	Speedup
7,3	1	76.62 ± 0.32	1.00	36.3	1	316.15 ± 1.26	1.00
7,3	2	53.58 ± 0.77	1.43	36.3	2	163.96 ± 1.09	1.93
7,3	4	24.36 ± 0.09	3.15	36.3	4	85.86 ± 0.48	3.68
7,3	8	13.2 ± 0.73	5.81	36.3	8	47.73 ± 0.37	6.62
7,3	16	11.47 ± 1.51	6.68	36.3	16	29.58 ± 0.33	10.69
7,3	32	11.03 ± 0.36	6.95	36.3	32	21.0 ± 0.81	15.05
12,7	1	127.09 ± 0.62	1.00	72,6	1	631.47 ± 2.93	1.00
12,7	2	62.51 ± 0.74	2.03	72,6	2	320.78 ± 1.44	1.97
12,7	4	35.54 ± 0.94	3.58	72,6	4	165.39 ± 1.26	3.82
12,7	8	22.06 ± 0.27	5.76	72,6	8	87.72 ± 1.43	7.20
12,7	16	15.4 ± 0.23	8.25	72,6	16	47.79 ± 0.77	13.21
12,7	32	13.12 ± 0.36	9.69	72,6	32	30.25 ± 0.39	20.87
18,2	1	178.42 ± 1.77	1.00	145,3	1	1246.22 ± 6.4	1.00
18,2	2	85.75 ± 0.81	2.08	145,3	2	630.19 ± 2.88	1.98
18,2	4	47.25 ± 0.60	3.78	145,3	4	323.15 ± 1.47	3.86
18,2	8	29.45 ± 1.17	6.06	145,3	8	166.86 ± 1.12	7.47
18,2	16	20.19 ± 0.94	8.84	145,3	16	87.10 ± 0.91	14.31
18,2	32	14.56 ± 0.27	12.25	145,3	32	48.58 ± 0.50	25.65

Dataset (GB)	Nodes	Time	Speedup
290,9	1	2491.89 ± 9.98	1.00
290,9	2	1256.70 ± 2.65	1.98
290,9	4	633.24 ± 0.02	3.94
290,9	8	323.98 ± 0.99	7.69
290,9	16	167.09 ± 0.98	14.91
290,9	32	89.13 ± 0.60	27.96

MDA by reducing variability between materials—as seen in both the SAD and RMSE values. Nevertheless, it falls short of DPGD in overall metrics, particularly in the soil and tree categories.

The proposed DPGD method again demonstrates the best overall performance, with the lowest Mean SAD (0.0513) and lowest Mean RMSE (0.2602) among all three methods. Although it does not provide the best SAD for any single material, its values are consistently close to the best. Regarding the soil (0.0455), its SAD is just 0.0051 points above the SAD of MDA. Also, focusing on the tree category (0.0284), it is only 0.0065 points above MDA. Finally, in the water class (0.0799), it reaches a value 0.0011 points above the NMDA result. This performance confirms the strength of DPGD in delivering low and stable errors across all materials, without major outliers. The absence of large SAD values, contrary to the MDA, is a key reason behind its superior Mean SAD and RMSE. The combination of accuracy and stability makes DPGD particularly suitable for practical unmixing tasks where varied land-cover classes (such as vegetation, soil, and water) must be handled uniformly.

E. Efficiency Evaluation

To further evaluate the efficiency and scalability of the proposed DPGD method, we have conducted a comprehensive set of experiments on the synthetic Samson dataset, progressively increasing the dataset size to 7.3, 12.7, 18.2, 36.3, 72.6, 145.3, and 290.9 GB. These experiments were executed on the distributed computing infrastructure detailed in Section IV-A, with the number of computing nodes varied systematically in powers of two, from 1 to 32, to simulate realistic deployments. Instead of increasing the number of computing nodes indefinitely, we adopted a dual scalability strategy, where both the dataset size and the number of nodes are varied systematically. This enables us to capture the saturation behavior of the algorithm for different workloads and to demonstrate the practical limits of parallelization in a cloud environment. For smaller datasets, the gains from adding more nodes tend to diminish (e.g., only a $0.27\times$ improvement from 16 to 32 nodes for the 7.3-GB dataset). However, as the data size increases, the efficiency of node utilization improves significantly, reaching a speedup of 27.96 with 32 nodes for the 290.9-GB dataset. These experiments illustrate how scalability behaves under realistic cloud deployment conditions, balancing workload size, node count, and resource costs. While our tests were limited to 32 nodes and datasets up to 290.9 GB, the observed trends align with established scalability theory: diminishing returns appear for small workloads, while larger datasets continue to benefit from additional resources.

To quantitatively assess the scalability of DPGD, we measured both the mean execution time (along with its standard deviation) and the achieved speedup as the number of nodes increased. These two metrics offer complementary insights: execution time reveals the raw computational efficiency for each configuration, while speedup indicates the parallel scalability and how well the method leverages additional computational resources. Detailed results, presented in Table III, allow us to analyze the behavior of the proposed DPGD across different data volumes and system scales.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 09,2026 at 11:45:48 UTC from IEEE Xplore. Restrictions apply.

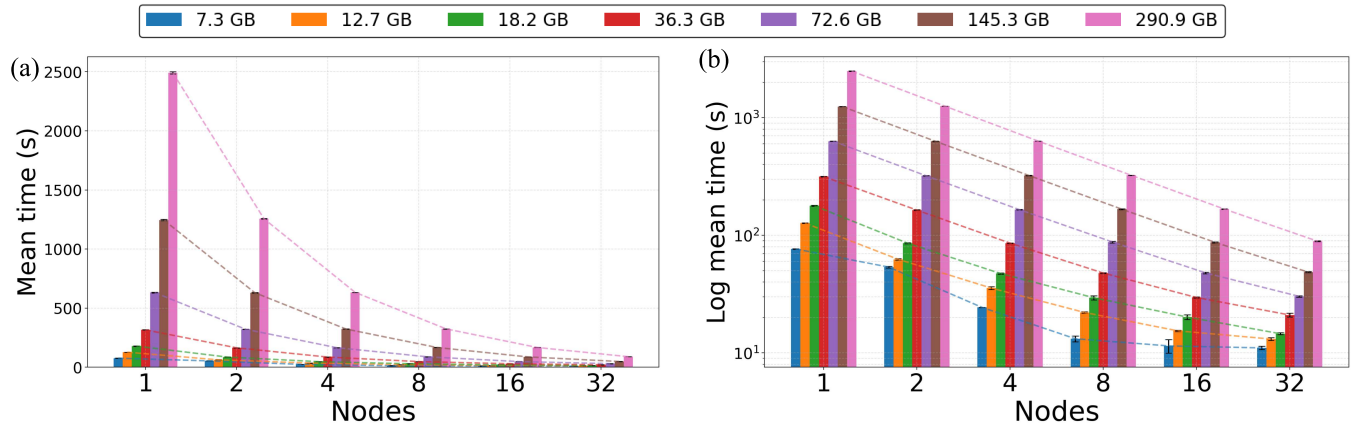


Fig. 3. (a) Comparison of mean execution time with standard deviation in linear and (b) logarithmic scale across all datasets and node configurations.

1) Runtime Analysis: Fig. 3 illustrates the relationship between the number of compute nodes and the execution time of the proposed DPGD method across varying dataset sizes (from 7.3 to 290.9 GB). In both plots, error bars indicate the standard deviation over five independent runs. As expected, increasing the number of nodes leads to a substantial reduction in execution time, demonstrating the method's ability to exploit distributed parallelism efficiently. The logarithmic view further highlights consistent scaling trends and low variability across configurations.

As reported in Table III, the execution time for the medium-to-large 72.6-GB dataset decreases from 631.47 s with a single node to 30.25 s using 32 nodes. This represents an approximate 95.2% reduction in runtime, corresponding to a 20.87× speedup, which closely approaches the ideal scalability target. Furthermore, as dataset sizes increase, the benefits of distributed processing become more apparent. For example, the 145.3-GB dataset demonstrates a steady decrease from 1246.22 s (1 node) to 48.58 s (32 nodes), with a 25.65× speedup, indicating excellent scalability even for datasets beyond 100 GB. Even more impressively, the largest dataset tested, measuring 290.9 GB, sees its runtime reduced from 2491.89 s on one node to 89.13 s on 32 nodes, achieving a 27.96× speedup. These results highlight the robustness and scalability of DPGD in processing medium, large, and very large hyperspectral datasets. This behavior is expected, as larger datasets provide greater opportunities for parallelism and reduce the relative impact of coordination and communication overheads. When data volume is sufficiently high, the computational workload can be more evenly distributed across the available nodes, allowing the parallel execution model to approach near-linear speedup under well-balanced conditions.

The trend is consistent for medium datasets, such as the 36.3- and 18.2-GB datasets, which show speedups of 15.05× and 12.25×, respectively, when scaled up to 32 nodes. However, for smaller datasets, such as 7.3 GB, the scalability gains begin to plateau. The execution time reduces from 76.62 s on a single node to 11.03 s on 32 nodes, yielding a speedup of 6.95×. While this still reflects a notable improvement, the relative gains are less pronounced compared to those

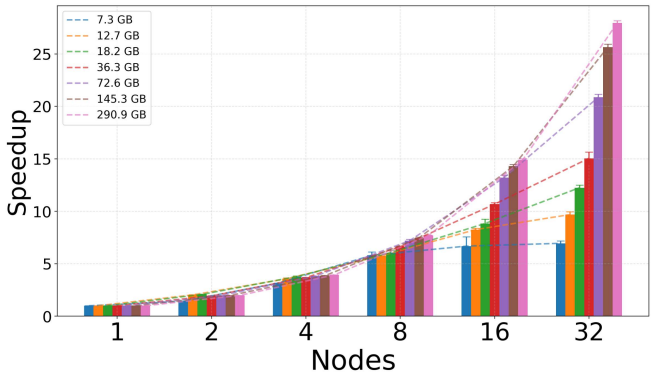


Fig. 4. Observed speedup of the proposed DPGD method across different dataset sizes.

achieved with larger datasets. This trend can be attributed to the communication, synchronization, and data partitioning overheads inherent in distributed systems. When the computational workload per node becomes small, these overheads start to dominate, limiting the effectiveness of adding more nodes.

2) Speedup Analysis: Fig. 4 evaluates the speedup achieved by the proposed DPGD method comparing between various dataset sizes as the number of computing nodes increases. The observed performance is compared against the theoretical ideal linear speedup, which assumes perfect parallelism without overhead. The results show that DPGD exhibits near-linear speedup behavior, demonstrating its strong scalability characteristics on a Spark-based cluster with up to 32 nodes and datasets up to 290.9 GB.

In the case of the largest 290.9-GB dataset, the method reaches a speedup of 27.96× using 32 nodes, i.e., approximately 87.4% of the ideal 32× performance. This indicates that the computational load is effectively partitioned and parallelized, with minimal idle time and excellent utilization of the distributed computing infrastructure. Similarly, for the 145.3-GB dataset, DPGD achieves a speedup of 25.65× under the same configuration, corresponding to 80.2% of the ideal linear scaling. A comparable trend is observed for the

Authorized licensed use limited to: Hohai University Library. Downloaded on November 09,2026 at 11:45:48 UTC from IEEE Xplore. Restrictions apply.

72.6- and 36.3-GB datasets, which attain speedups of 20.87× and 15.05×, respectively, with 32 nodes.

In contrast, for smaller datasets, such as 7.3 and 12.7 GB, the speedup curve begins to flatten beyond 8–16 nodes. For instance, the 7.3-GB dataset achieves a speedup of only 6.95× on 32 nodes, i.e., it achieves just 21.7% of the theoretical maximum. This behavior is produced by the overheads inherent in distributed computation, such as task scheduling, network communication, data shuffling, and synchronization. When the data size is small, these overheads become significant relative to the actual computation, limiting the effectiveness of adding more nodes.

While the near-linear speedup observed in the experiments is consistent with the general capabilities of Spark for distributed data processing, it should be noted that such scalability is not universally attainable across all types of algorithms. The DPGD algorithm is explicitly designed to avoid global synchronization and excessive communication, making it well-suited to Spark's execution model. Unlike standard Spark workloads, including flat map-reduce operations or SQL queries assessed in benchmarks such as TPC-DS [75] and BigDataBench [76], the current pipeline is characterized by high-dimensional geometric processing and iterative affine hull estimation, which are inherently more difficult to scale efficiently. Prior studies have shown that Spark's performance deteriorates significantly under iterative, memory-bound, or communication-heavy patterns [77], [78]. Thus, the strong scalability observed in this study reflects both the architectural advantages of Spark and the parallel-aware design of the proposed method.

V. Conclusion

In this article, we introduce a novel distributed parallel hyperspectral unmixing approach called DPGD, which leverages geometric distances within a Spark framework. The proposed DPGD significantly enhances the capability to process gigabyte-scale hyperspectral datasets by addressing the key challenges of computational efficiency and scalability faced by traditional hyperspectral unmixing methods. The proposed approach, based on innovative geometric distance measures, not only improves the accuracy of endmember estimation but also leverages distributed computing to accelerate processing and enable the efficient handling of hyperspectral datasets up to 290.9 GB in size and cluster configurations of up to 32 nodes. Experimental results on the Samson and Cuprite datasets demonstrate that the proposed method outperforms current state-of-the-art techniques in both accuracy and processing efficiency. We explicitly limit our claims to this tested range and note that scalability beyond 32 nodes has not been empirically validated within this study.

Future work will explore the potential benefits of decentralizing the control logic currently managed by the Spark manager node. This includes evaluating alternative configurations that support distributed or replicated coordination and investigating consensus-based orchestration strategies inspired by production systems such as Redis replication or Apache ZooKeeper, with the aim of improving scalability and runtime resilience in fault-prone or elastic cloud environments.

Additionally, we plan to benchmark DPGD against other parallel or distributed unmixing algorithms, such as those based on MPI, Hadoop, or GPU acceleration, to further assess its relative performance. While such implementations are often tied to specific architectures or lack publicly available code, we recognize that integrating them into a comparable experimental framework would provide a broader evaluation of scalability and accuracy. To this end, future experiments will include the use of larger cloud clusters with more than 32 nodes, when resources are available, in order to empirically establish the saturation point of scalability and provide further guidance for large-scale deployments.

Acknowledgment

The BBVA Foundation accepts no responsibility for the opinions, statements, and contents included in the project and/or the results thereof, which are entirely the responsibility of the authors.

References

[1] T. C. Vance, T. Huang, and K. A. Butler, "Big data in Earth science: Emerging practice and promise," *Science*, vol. 383, no. 6688, p. 9607, Mar. 2024.

[2] European Space Agency.(2022). *ESA Strategy 2040—In Depth*. Accessed: Jul. 9, 2025. [Online]. Available: https://esamultimedia.esa.int/docs/corporate/ESA_Strategy_2040_InDepth.pdf

[3] E. Kanagaraj, L. M. Kamarudin, A. Zakaria, R. Gunasagaran, and A. Y. M. Shakaff, "Cloud-based remote environmental monitoring system with distributed WSN weather stations," in *Proc. IEEE SENSORS*, Nov. 2015, pp. 1–4.

[4] R. T. Eguchi, C. K. Huyck, S. Ghosh, and B. J. Adams, "The application of remote sensing technologies for disaster management," in *Proc. 14th World Conf. Earthq. Eng.*, vol. 17, 2008, pp. 1–17.

[5] K. Kaku, "Satellite remote sensing for disaster management support: A holistic and staged approach based on case studies in sentinel Asia," *Int. J. Disaster Risk Reduction*, vol. 33, pp. 417–432, Feb. 2019.

[6] A. Comber, P. Fisher, C. Brunsdon, and A. Khmag, "Spatial analysis of remote sensing image classification accuracy," *Remote Sens. Environ.*, vol. 127, pp. 237–246, Dec. 2012.

[7] J. M. Haut, M. E. Paoletti, S. Moreno-Álvarez, J. Plaza, J.-A. Rico-Gallego, and A. Plaza, "Distributed deep learning for remote sensing data interpretation," *Proc. IEEE*, vol. 109, no. 8, pp. 1320–1349, Aug. 2021.

[8] F. Liang, Z. Zhang, H. Lu, V. C. M. Leung, Y. Guo, and X. Hu, "Communication-efficient large-scale distributed deep learning: A comprehensive survey," 2024, *arXiv:2404.06114*.

[9] N. Aburaed, M. Q. Alkhatib, S. Marshall, J. Zabalza, and H. Al Ahmad, "A review of spatial enhancement of hyperspectral remote sensing imaging techniques," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 16, pp. 2275–2300, 2023.

[10] A. Carbone et al., "Prisma second generation concept design for optimized data acquisition and responsiveness in disaster management," in *Proc. 9th Int. Conf. Cartography GIS*, 2024, pp. 789–800.

[11] J. Nieke et al., "The Copernicus hyperspectral imaging mission for the environment (CHIME): An overview of its mission, system and planning status," *Proc. SPIE*, vol. 12729, pp. 21–40, Mar. 2023.

[12] V. Chandran Suja, J. Sentmanat, G. Hofmann, C. Scales, and G. G. Fuller, "Hyperspectral imaging for dynamic thin film interferometry," *Sci. Rep.*, vol. 10, no. 1, p. 11378, Jul. 2020.

[13] B. Borguet, V. Moreau, S. Santandrea, J. Versluys, and A. Bourdoux, "The challenges of broadband performances within a compact imaging spectrometer: The ELOIS solution," *Proc. SPIE*, vol. 11852, pp. 572–582, Jun. 2021.

[14] J. Blommaert et al., "CSIMBA: Towards a smart-spectral cubesat constellation," in *Proc. IEEE Int. Geosci. Remote Sens. Symp.*, Jul. 2019, pp. 4614–4617.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 09,2026 at 11:45:48 UTC from IEEE Xplore. Restrictions apply.

[15] S. Zhang, Y. Su, X. Xu, J. Li, C. Deng, and A. Plaza, "Recent advances in hyperspectral unmixing using sparse techniques and deep learning," in *Hyperspectral Image Analysis: Advances in Machine Learning and Signal Processing*. Springer, 2020, pp. 377–405.

[16] C. Shi and L. Wang, "Incorporating spatial information in spectral unmixing: A review," *Remote Sens. Environ.*, vol. 149, pp. 70–87, Jun. 2014.

[17] A. Min, Z. Guo, H. Li, and J. Peng, "JMnet: Joint metric neural network for hyperspectral unmixing," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, 2022, Art. no. 5505412.

[18] X. Tao, M. E. Paoletti, J. M. Haut, P. Ren, J. Plaza, and A. Plaza, "Endmember estimation with maximum distance analysis," *Remote Sens.*, vol. 13, no. 4, p. 713, Feb. 2021.

[19] X. Tao et al., "Endmember estimation from hyperspectral images using geometric distances," *IEEE Geosci. Remote Sens. Lett.*, vol. 19, pp. 1–5, 2022.

[20] E. Torti, G. Danese, F. Leporati, and A. Plaza, "A hybrid CPU–GPU real-time hyperspectral unmixing chain," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 9, no. 2, pp. 945–951, Feb. 2016.

[21] P. Zheng et al., "A parallel unmixing-based content retrieval system for distributed hyperspectral imagery repository on cloud computing platforms," *Remote Sens.*, vol. 13, no. 2, p. 176, Jan. 2021.

[22] R. O. Green et al., "Imaging spectroscopy and the airborne visible/infrared imaging spectrometer (AVIRIS)," *Remote Sens. Environ.*, vol. 65, no. 3, pp. 227–248, 1998.

[23] L. Hamlin et al., "Imaging spectrometer science measurements for terrestrial ecology: AVIRIS and new developments," in *Proc. Aerosp. Conf.*, Mar. 2011, pp. 1–7.

[24] R. O. Green et al., "Airborne visible/infrared imaging spectrometer 3 (AVIRIS-3)," in *Proc. IEEE Aerosp. Conf. (AERO)*, Mar. 2022, pp. 1–10.

[25] L. Candela, R. Formaro, R. Guarini, R. Loizzo, F. Longo, and G. Varacalli, "The PRISMA mission," in *Proc. IEEE Int. Geosci. Remote Sens. Symp. (IGARSS)*, Jul. 2016, pp. 253–256.

[26] T. Storch et al., "The EnMAP imaging spectroscopy mission towards operations," *Remote Sens. Environ.*, vol. 294, Jan. 2023, Art. no. 113632.

[27] M. Rast et al., "Copernicus hyperspectral imaging mission for the environment: Mission requirements document," Eur. Space Agency (ESA), Noordwijk, The Netherlands, Tech. Rep.ESA-EOPSM-CHIMMRD-3216, 2019.

[28] M. Guelman and F. Ortenberg, "Small satellite's role in future hyperspectral Earth observation missions," *Acta Astronautica*, vol. 64, nos. 11–12, pp. 1252–1263, Jun. 2009.

[29] A. Piro et al., "HYBRIS: Analysis and design of a hyper-spectral cubesat mission for multiple remote sensing applications and Earth observation synergies," in *Proc. 4S Symp.*, 2018, pp. 1–10.

[30] M. E. Grøtte et al., "Ocean color hyperspectral remote sensing with high resolution and low latency—The HYPSO-1 CubeSat mission," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, 2022, Art. no. 1000619.

[31] Y. Ma et al., "Remote sensing big data computing: Challenges and opportunities," *Future Gener. Comput. Syst.*, vol. 51, pp. 47–60, Oct. 2015.

[32] M. Chi, A. Plaza, J. A. Benediktsson, Z. Sun, J. Shen, and Y. Zhu, "Big data for remote sensing: Challenges and opportunities," *Proc. IEEE*, vol. 104, no. 11, pp. 2207–2219, Nov. 2016.

[33] P. Liu, "A survey of remote-sensing big data," *Frontiers Environ. Sci.*, vol. 3, p. 45, Jun. 2015.

[34] S. Thudumu, P. Branch, J. Jin, and J. Singh, "A comprehensive survey of anomaly detection techniques for high dimensional big data," *J. Big Data*, vol. 7, no. 1, p. 42, Dec. 2020.

[35] J. A. G. Jaramago, M. E. Paoletti, J. M. Haut, R. Fernandez-Beltran, A. Plaza, and J. Plaza, "GPU parallel implementation of dual-depth sparse probabilistic latent semantic analysis for hyperspectral unmixing," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 12, no. 9, pp. 3156–3167, Sep. 2019.

[36] J. M. Bioucas-Dias et al., "Hyperspectral unmixing overview: Geometrical, statistical, and sparse regression-based approaches," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 5, no. 2, pp. 354–379, Apr. 2012.

[37] R. Heylen, M. Parente, and P. Gader, "A review of nonlinear hyperspectral unmixing methods," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 7, no. 6, pp. 1844–1868, Jun. 2014.

[38] W. Xia, X. Liu, B. Wang, and L. Zhang, "Independent component analysis for blind unmixing of hyperspectral imagery with additional constraints," *IEEE Trans. Geosci. Remote Sens.*, vol. 49, no. 6, pp. 2165–2179, Jun. 2011.

[39] J. M. P. Nascimento and J. M. Bioucas-Dias, "Dependent component analysis: A hyperspectral unmixing algorithm," in *Proc. Iberian Conf. Pattern Recognit. Image Anal.*, 2007, pp. 612–619.

[40] C. F. Caiafa, E. Salerno, A. N. Proto, and L. Fiumi, "Blind spectral unmixing by local maximization of non-Gaussianity," *Signal Process.*, vol. 88, no. 1, pp. 50–68, Jan. 2008.

[41] H. Liu, Y. Lu, Z. Wu, Q. Du, J. Chanussot, and Z. Wei, "Spectral variability Bayesian unmixing for hyperspectral sequence in wavelet domain," *IEEE Trans. Geosci. Remote Sens.*, vol. 61, 2023, Art. no. 5522719.

[42] M.-D. Iordache, J. M. Bioucas-Dias, and A. Plaza, "Sparse unmixing of hyperspectral data," *IEEE Trans. Geosci. Remote Sens.*, vol. 49, no. 6, pp. 2014–2039, Jun. 2011.

[43] M.-D. Iordache, J. M. Bioucas-Dias, and A. Plaza, "Total variation spatial regularization for sparse hyperspectral unmixing," *IEEE Trans. Geosci. Remote Sens.*, vol. 50, no. 11, pp. 4484–4502, Nov. 2012.

[44] M.-D. Iordache, J. M. Bioucas-Dias, and A. Plaza, "Collaborative sparse regression for hyperspectral unmixing," *IEEE Trans. Geosci. Remote Sens.*, vol. 52, no. 1, pp. 341–354, Jan. 2014.

[45] Z. Shi, T. Shi, M. Zhou, and X. Xu, "Collaborative sparse hyperspectral unmixing using $l_\{0\}$ norm," *IEEE Trans. Geosci. Remote Sens.*, vol. 56, no. 9, pp. 5495–5508, Sep. 2018.

[46] F. Zhu, Y. Wang, S. Xiang, B. Fan, and C. Pan, "Structured sparse method for hyperspectral unmixing," *ISPRS J. Photogramm. Remote Sens.*, vol. 88, pp. 101–118, Feb. 2014.

[47] T. Chen, Y. Liu, Y. Zhang, B. Du, and A. Plaza, "Superpixel-based collaborative and low-rank regularization for sparse hyperspectral unmixing," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, 2022, Art. no. 5529216.

[48] X. Tao et al., "Fast orthogonal projection for hyperspectral unmixing," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, 2022, Art. no. 5523313.

[49] L. Ren, D. Hong, L. Gao, X. Sun, M. Huang, and J. Chanussot, "Orthogonal subspace unmixing to address spectral variability for hyperspectral image," *IEEE Trans. Geosci. Remote Sens.*, vol. 61, 2023, Art. no. 5501713.

[50] C.-I. Chang, C.-C. Wu, and C.-T. Tsai, "Random N-finder (N-FINDR) endmember extraction algorithms for hyperspectral imagery," *IEEE Trans. Image Process.*, vol. 20, no. 3, pp. 641–656, Mar. 2011.

[51] C.-I. Chang and C.-C. Wu, "Design and development of iterative pixel purity index," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 8, no. 6, pp. 2676–2695, Jun. 2015.

[52] J. M. P. Nascimento and J. M. B. Dias, "Vertex component analysis: A fast algorithm to unmix hyperspectral data," *IEEE Trans. Geosci. Remote Sens.*, vol. 43, no. 4, pp. 898–910, Apr. 2005.

[53] T.-H. Chan, C.-Y. Chi, Y.-M. Huang, and W.-K. Ma, "A convex analysis-based minimum-volume enclosing simplex algorithm for hyperspectral unmixing," *IEEE Trans. Signal Process.*, vol. 57, no. 11, pp. 4418–4432, Nov. 2009.

[54] J. Li, A. Agathos, D. Zaharie, J. M. Bioucas-Dias, A. Plaza, and X. Li, "Minimum volume simplex analysis: A fast algorithm for linear hyperspectral unmixing," *IEEE Trans. Geosci. Remote Sens.*, vol. 53, no. 9, pp. 5067–5082, Sep. 2015.

[55] L. Miao and H. Qi, "Endmember extraction from highly mixed data using minimum volume constrained nonnegative matrix factorization," *IEEE Trans. Geosci. Remote Sens.*, vol. 45, no. 3, pp. 765–777, Mar. 2007.

[56] G. Zhou, S. Xie, Z. Yang, J.-M. Yang, and Z. He, "Minimum-volume-constrained nonnegative matrix factorization: Enhanced ability of learning parts," *IEEE Trans. Neural Netw.*, vol. 22, no. 10, pp. 1626–1637, Oct. 2011.

[57] S. Bernabé, S. Sánchez, A. Plaza, S. López, J. A. Benediktsson, and R. Sarmiento, "Hyperspectral unmixing on GPUs and multi-core processors: A comparison," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 6, no. 3, pp. 1386–1398, Jun. 2013.

[58] Z. Li, Y. Altmann, J. Chen, S. Mclaughlin, and S. Rahardja, "Sparse linear spectral unmixing of hyperspectral images using expectation-propagation," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, 2022, Art. no. 5524313.

[59] Z. Li, J. Chen, M. M. Movania, and S. Rahardja, "GPU implementation of graph-regularized sparse unmixing with superpixel structures," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 16, pp. 3378–3389, 2023.

[60] V. A. A. Quirita, G. A. O. P. da Costa, and C. Beltrán, "A distributed N-FINDR cloud computing-based solution for endmembers extraction on large-scale hyperspectral remote sensing data," *Remote Sens.*, vol. 14, no. 9, p. 2153, Apr. 2022.

[61] C. González, J. Resano, D. Mozos, A. Plaza, and D. Valencia, "FPGA implementation of the pixel purity index algorithm for remotely sensed hyperspectral image analysis," *EURASIP J. Adv. Signal Process.*, vol. 2010, no. 1, Dec. 2010, Art. no. 969806.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 09,2026 at 11:45:48 UTC from IEEE Xplore. Restrictions apply.

[62] A. Barberis, G. Danese, F. Leporati, A. Plaza, and E. Torti, "Real-time implementation of the vertex component analysis algorithm on GPUs," *IEEE Geosci. Remote Sens. Lett.*, vol. 10, no. 2, pp. 251–255, Mar. 2013.

[63] X. Wu, B. Huang, A. Plaza, Y. Li, and C. Wu, "Real-time implementation of the pixel purity index algorithm for endmember identification on GPUs," *IEEE Geosci. Remote Sens. Lett.*, vol. 11, no. 5, pp. 955–959, May 2014.

[64] J. F. de Rosário, J. M. P. Nascimento, and M. Véstias, "FPGA-based architecture for hyperspectral endmember extraction," *Proc. SPIE*, vol. 9247, Jul. 2014, Art. no. 924703.

[65] W. Xiong, C.-I. Chang, C.-C. Wu, K. Kalpakis, and H. M. Chen, "Fast algorithms to implement N-FINDR for hyperspectral endmember extraction," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 4, no. 3, pp. 545–564, Sep. 2011.

[66] J. Li and J. M. Bioucas-Dias, "Minimum volume simplex analysis: A fast algorithm to unmix hyperspectral data," in *Proc. IEEE Int. Geosci. Remote Sens. Symp.*, Apr. 2008, pp. III-25–III-253.

[67] X. Zhang, Y. Wang, and T. Xue, "Quadratic clustering-based simplex volume maximization for hyperspectral endmember extraction," *Appl. Sci.*, vol. 12, no. 14, p. 7132, Jul. 2022.

[68] A. Rizaldy, A. J. Afifi, P. Ghamisi, and R. Gloaguen, "Improving mineral classification using multimodal hyperspectral point cloud data and multi-stream neural network," *Remote Sens.*, vol. 16, no. 13, p. 2336, Jun. 2024.

[69] H. Li et al., "Sparse unmixing guided adversarial attack for hyperspectral image classification," *IEEE Trans. Circuits Syst. Video Technol.*, early access, Aug. 25, 2025, doi: 10.1109/TCSVT.2025.3602604.

[70] S. Qin et al., "Multimodal uncertainty state-space fusion network with generative endmember modeling for hyperspectral unmixing," *Inf. Fusion*, vol. 127, Mar. 2026, Art. no. 103712.

[71] W. Gao, Y. Zhang, Y. Akoudad, and J. Chen, "MSSF-Net: A multimodal spectral–spatial feature fusion network for hyperspectral unmixing," *IEEE Trans. Geosci. Remote Sens.*, vol. 63, 2025, Art. no. 5511515.

[72] J. Chen et al., "TCCU-Net: Transformer and CNN collaborative unmixing network for hyperspectral image," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 17, pp. 8073–8089, 2024.

[73] J. Zou, H. Qu, and P. Zhang, "Conventional to deep learning methods for hyperspectral unmixing: A review," *Remote Sens.*, vol. 17, no. 17, p. 2968, Aug. 2025.

[74] M. Zaharia, M. Chowdhury, M. J. Franklin, S. Shenker, and I. Stoica, "Spark: Cluster computing with working sets," in *Proc. 2nd USENIX Workshop Hot Topics Cloud Comput. (HotCloud 10)*, 2010, p. 10.

[75] (2023). *TPC Benchmark DS (Decision Support)*. [Online]. Available: http://www.tpc.org/tpcds/

[76] L. Wang et al., "BigDataBench: A big data benchmark suite from Internet services," *IEEE Trans. Parallel Distrib. Syst.*, vol. 26, no. 8, pp. 2091–2101, Aug. 2015.

[77] H. Karau and A. Konwinski, "Learning spark performance: Automated tuning through workload classification," in *Proc. Strata+Hadoop World*, 2015.

[78] M. Zaharia, T. Das, H. Li, T. Hunter, S. Shenker, and I. Stoica, "Discretized streams: Fault-tolerant streaming computation at scale," *Commun. ACM*, vol. 59, no. 11, pp. 93–102, 2016.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 09,2026 at 11:45:48 UTC from IEEE Xplore. Restrictions apply.